10장. 예외처리

이것이 자바다



Contents

- 1절. 예외와 예외 클래스
- 2절. 실행 예외
- 3절. 예외 처리 코드
- 4절. 예외 종류에 따른 처리 코드
- 5절. 자동 리소스 닫기
- 6절. 예외 처리 떠넘기기
- 7절. 사용자 정의 예외와 예외 발생



1절. 예외와 예외 클래스

❖ 오류의 종류

- 에러(Error)
  - 하드웨어의 잘못된 동작 또는 고장으로 인한 오류
  - 에러가 발생되면 프로그램 종료
  - 정상 실행 상태로 돌아갈 수 없음
- 예외(Exception)
  - 사용자의 잘못된 조작 또는 개발자의 잘못된 코딩으로 인한 오류
  - 예외가 발생되면 프로그램 종료
  - 예외 처리 추가하면 정상 실행 상태로 돌아갈 수 있음



1절. 예외와 예외 클래스

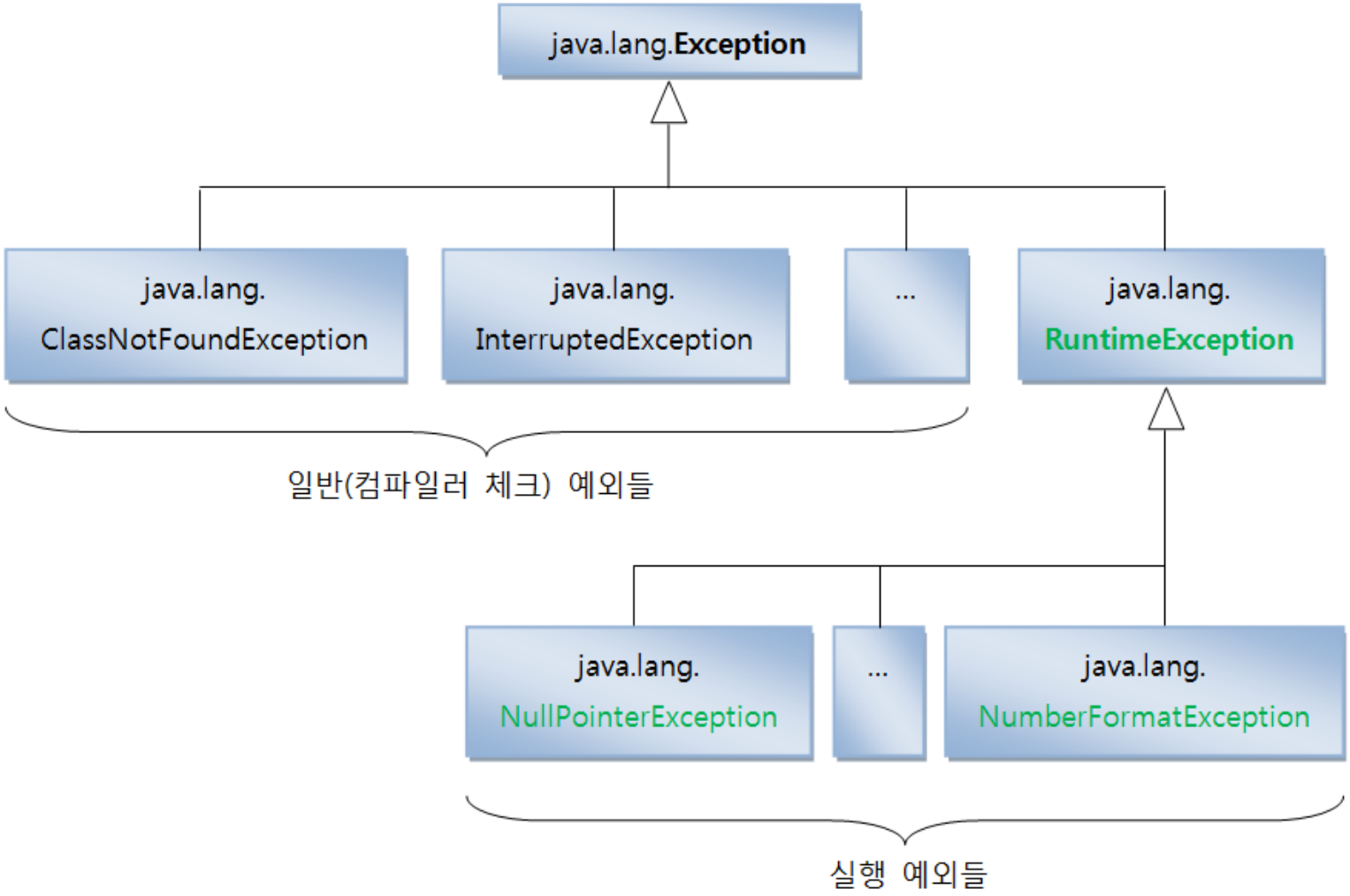
❖ 예외의 종류

- 일반(컴파일 체크) 예외(Exception)
  - 예외 처리 코드 없으면 컴파일 오류 발생
- 실행 예외(RuntimeException)
  - 예외 처리 코드를 생략하더라도 컴파일이 되는 예외
  - 경험 따라 예외 처리 코드 작성 필요



1절. 예외와 예외 클래스

❖ 예외 클래스



2절. 실행 예외(RuntimeException)

❖ NullPointerException

- 객체 참조가 없는 상태
  - null 값 갖는 참조변수로 객체 접근 연산자인 도트(.) 사용했을 때 발생

```
String data = null;
System.out.println(data.toString());
```

❖ ArrayIndexOutOfBoundsException (p.424~425)

- 배열에서 인덱스 범위 초과하여 사용할 경우 발생

```
public static void main(String[] args) {
   String data1 = args[0];
   String data2 = args[1];

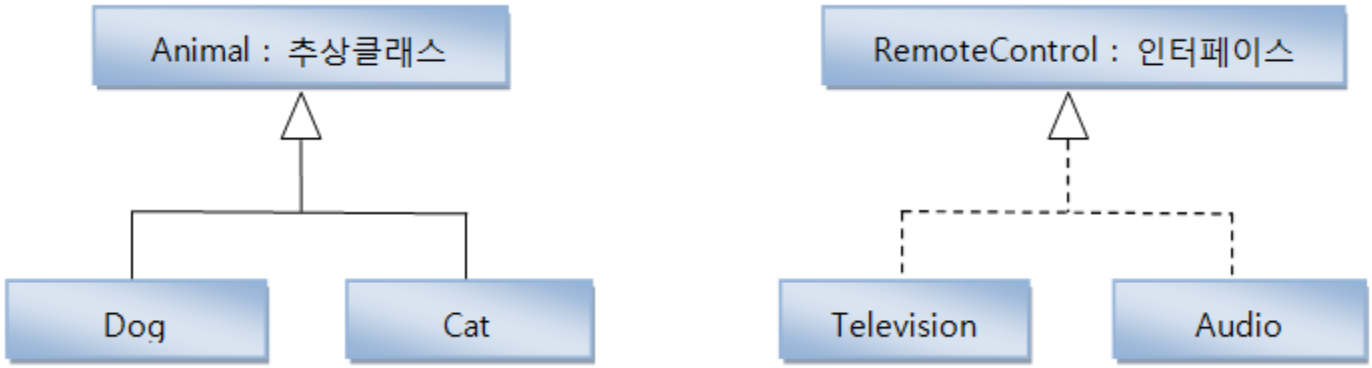
   System.out.println("args[0]: " + data1);
   System.out.println("args[1]: " + data2);
}
```



2절. 실행 예외(RuntimeException)

❖ ClassCastException

- 타입 변환이 되지 않을 경우 발생



- 정상 코드

```
Animal animal = new Dog();
Dog dog = (Dog) animal;
```

```
RemoteControl rc = new Television();
Television tv = (Television) rc;
```

- 예외 발생 코드

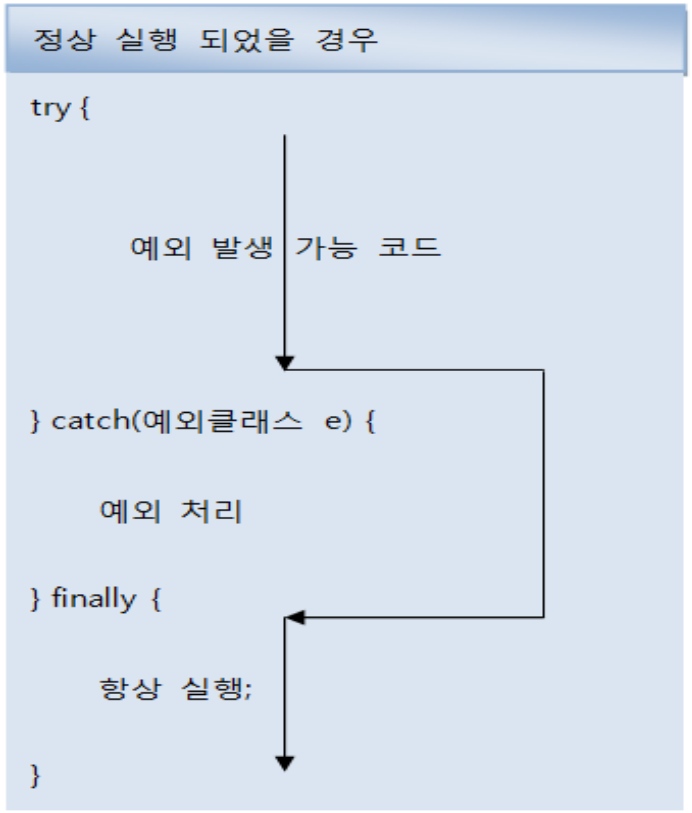
```
Animal animal = new Dog();
Cat cat = (Cat) animal;
```

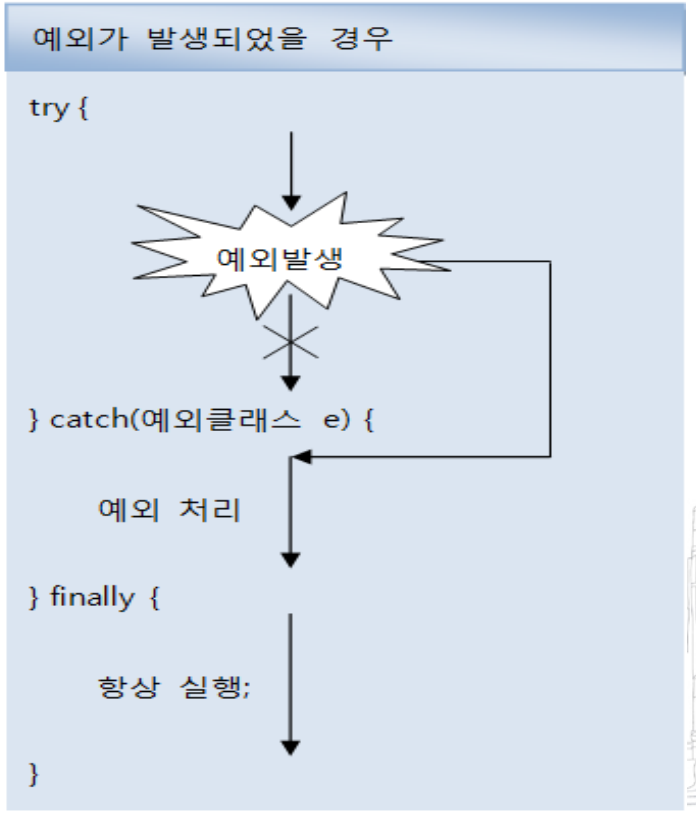
```
RemoteControl rc = new Television();
Audio audio = (Audio) rc;
```

3절. 예외 처리 코드(try-catch-finally)

❖ 예외 처리 코드 (p.429~432)

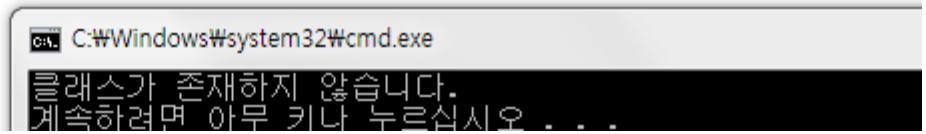
- 예외 발생시 프로그램 종료 막고, 정상 실행 유지할 수 있도록 처리
  - 일반 예외: 반드시 작성해야 컴파일 가능
  - 실행 예외: 컴파일러가 체크해주지 않으며 개발자 경험 의해 작성
- try – catch – finally 블록 이용해 예외 처리 코드 작성





TryCatchFinallyExample.java_일반 예외 처리

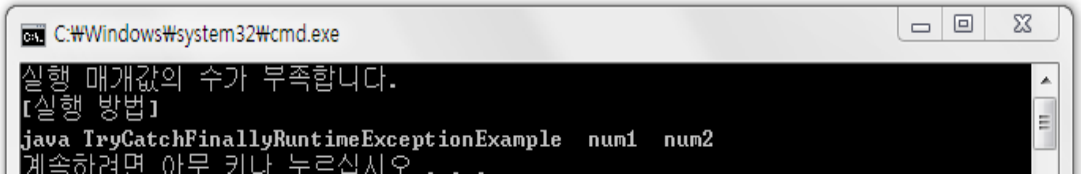
```
1 public class TryCatchFinallyExample {
2     public static void main(String[] args) {
3         try {
4             Class clazz = Class.forName("java.lang.String2");
5         } catch(ClassNotFoundException e) {
6             System.out.println("클래스가 존재하지 않습니다.");
7         }
8     }
9 }
```



TryCatchFinallyRuntimeExceptionExample.java_일반 예외 처리

```java
public class TryCatchFinallyRuntimeExceptionExample {
    public static void main(String[] args) {
        String data1 = null;
        String data2 = null;
        try {
            data1 = args[0];
            data2 = args[1];
        } catch(ArrayIndexOutOfBoundsException e) {
            System.out.println("실행 매개값의 수가 부족합니다.");
            System.out.println("[실행 방법]");
            System.out.println("java TryCatchFinallyRuntimeExceptionExample  num1  num2");
            return;
        }

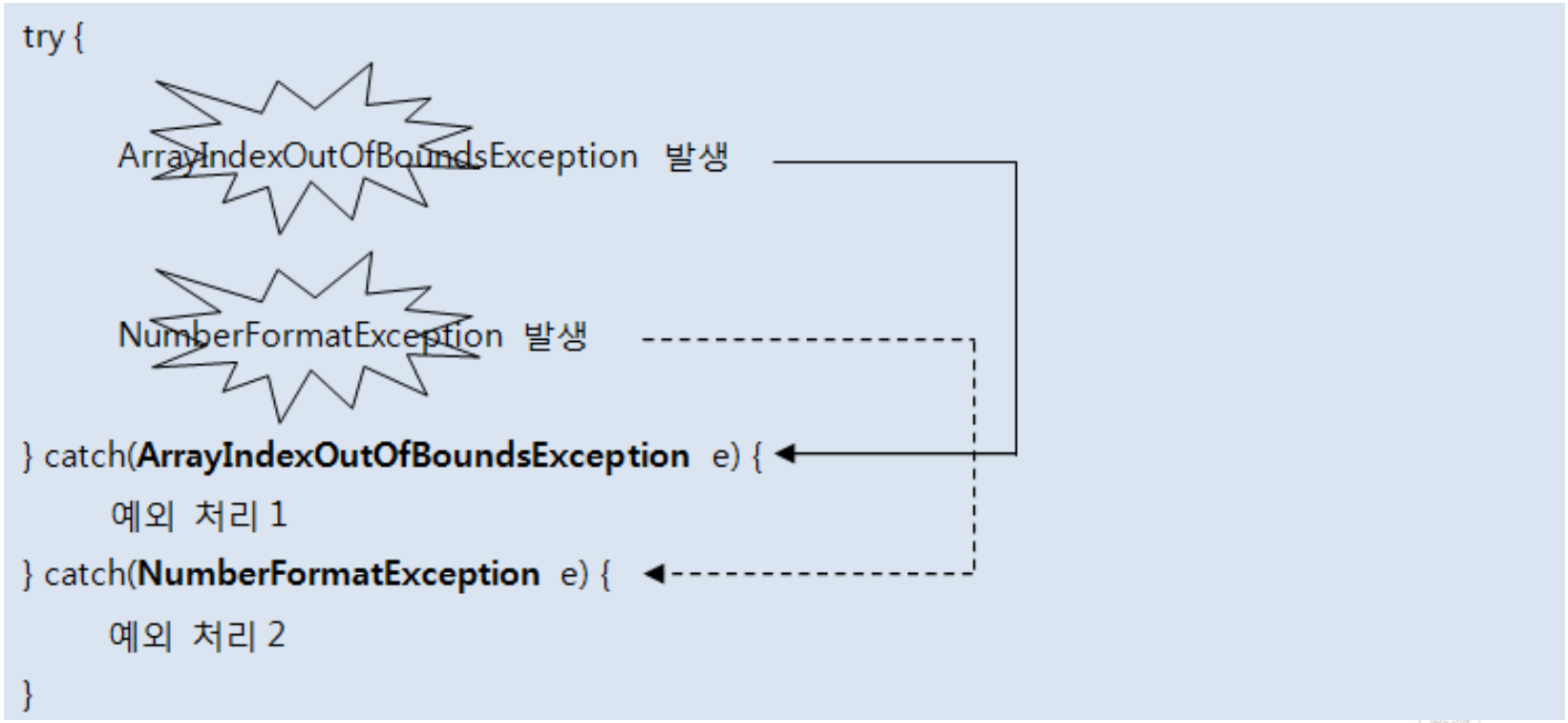
        try {
            int value1 = Integer.parseInt(data1);
            int value2 = Integer.parseInt(data2);
            int result = value1 + value2;
            System.out.println(data1 + "+" + data2 + "=" + result);
        } catch(NumberFormatException e) {
            System.out.println("숫자로 변환할 수 없습니다.");
        } finally {
            System.out.println("다시 실행하세요.");
        }
    }
}
```



4절. 예외 종류에 따른 처리 코드

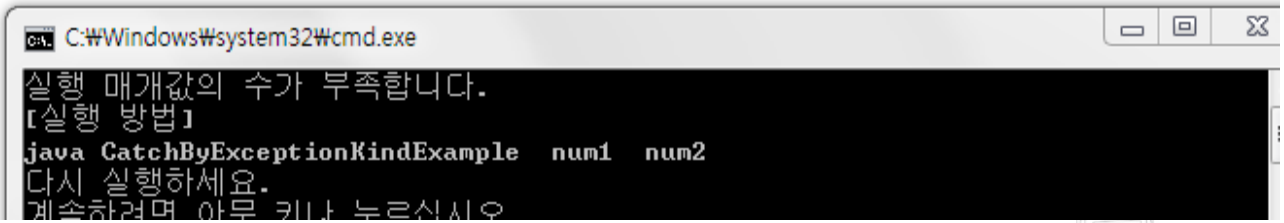
❖ 다중 catch

- 예외 별로 예외 처리 코드 다르게 구현



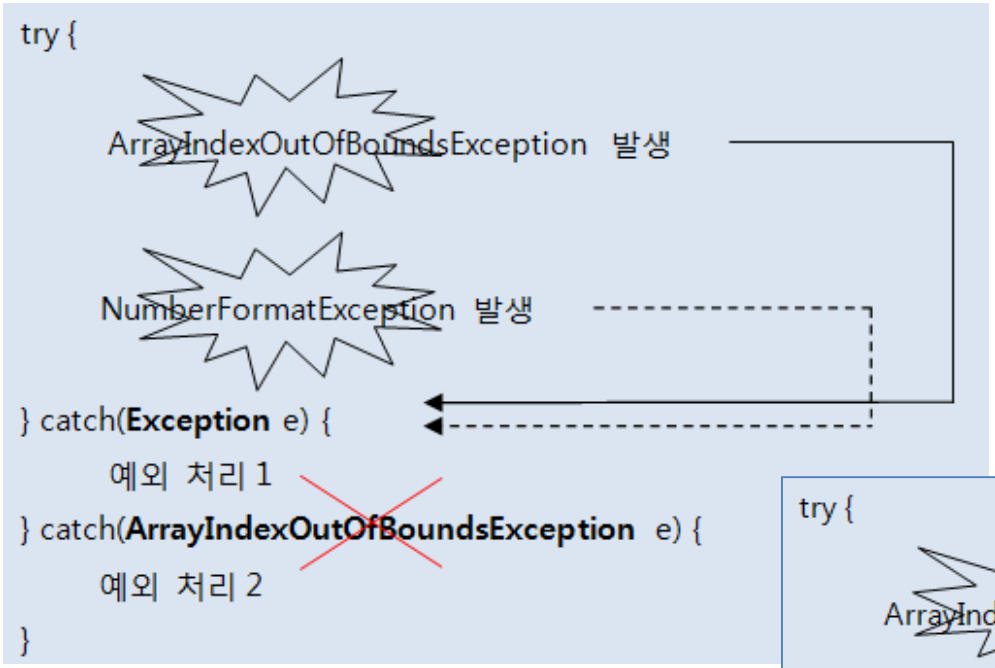
CatchByExceptionKindExample.java_다중 catch

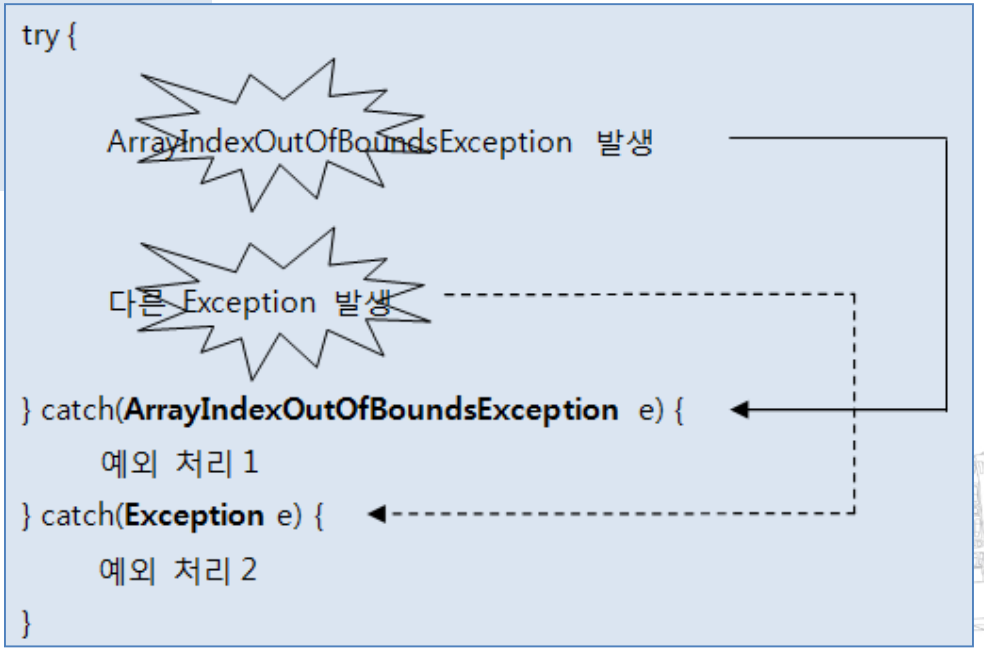
```java
public class CatchByExceptionKindExample {
    public static void main(String[] args) {
        try {
            String data1 = args[0];
            String data2 = args[1];
            int value1 = Integer.parseInt(data1);
            int value2 = Integer.parseInt(data2);
            int result = value1 + value2;
            System.out.println(data1 + "+" + data2 + "=" + result);
        } catch(ArrayIndexOutOfBoundsException e) {
            System.out.println("실행 매개값의 수가 부족합니다.");
            System.out.println("[실행 방법]");
            System.out.println("java CatchByExceptionKindExample  num1  num2");
        } catch(NumberFormatException e) {
            System.out.println("숫자로 변환할 수 없습니다.");
        } finally {
            System.out.println("다시 실행하세요.");
        }
    }
}
```



4절. 예외 종류에 따른 처리 코드

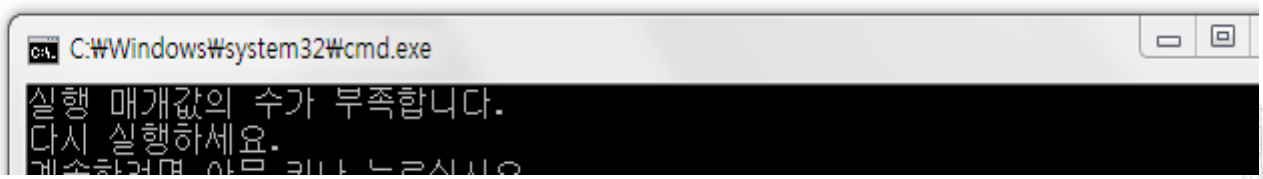
❖ catch 순서 – 상위 클래스가 위에 위치해야





CatchOrderExample.java_catch 블록의 순서

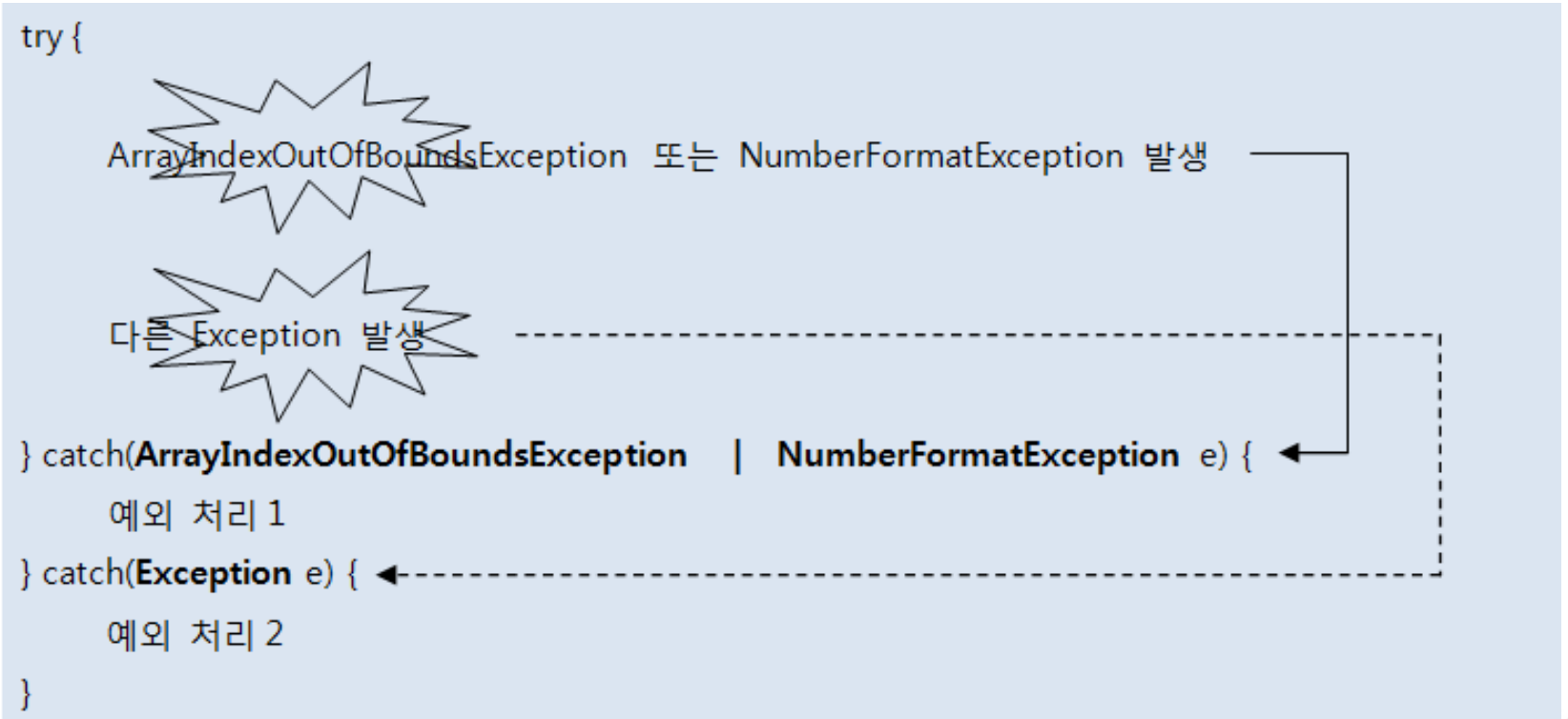
```
1  public class CatchOrderExample {
2      public static void main(String[] args) {
3          try {
4              String data1 = args[0];
5              String data2 = args[1];
6              int value1 = Integer.parseInt(data1);
7              int value2 = Integer.parseInt(data2);
8              int result = value1 + value2;
9              System.out.println(data1 + "+" + data2 + "=" + result);
10         } catch(ArrayIndexOutOfBoundsException e) {
11             System.out.println("실행 매개값의 수가 부족합니다.");
12         } catch(Exception e) {
13             System.out.println("실행에 문제가 있습니다.");
14         } finally {
15             System.out.println("다시 실행하세요.");
16         }
17     }
18 }
19
```



4절. 예외 종류에 따른 처리 코드

❖ 멀티(multi) catch (p.436~437)

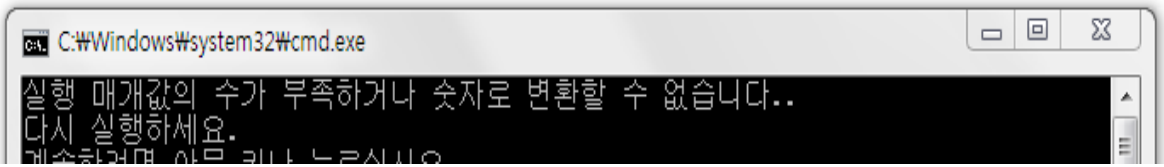
- 자바 7부터는 하나의 catch 블록에서 여러 개의 예외 처리 가능
  - 동일하게 처리하고 싶은 예외를 |로 연결



MultiCatchExample.java_Multi catch

```java
import java.io.IOException;
import java.net.Socket;
import java.net.UnknownHostException;

public class MultiCatchExample {
    public static void main(String[] args) {
        try {
            String data1 = args[0];
            String data2 = args[1];
            int value1 = Integer.parseInt(data1);
            int value2 = Integer.parseInt(data2);
            int result = value1 + value2;
            System.out.println(data1 + "+" + data2 + "=" + result);
        } catch(ArrayIndexOutOfBoundsException | NumberFormatException e) {
            System.out.println("실행 매개값의 수가 부족하거나 숫자로 변환할 수 없습니다..");
        } catch(Exception e) {
            System.out.println("알수 없은 예외 발생");
        } finally {
            System.out.println("다시 실행하세요.");
        }
    }
}
```



5절. 자동 리소스 닫기

❖ try-with-resources (p.438~440)

- 예외 발생 여부와 상관 없음
- 사용했던 리소스 객체의 close() 메소드 호출해 리소스 닫음
- 리소스 객체
  - 각종 입출력스트림, 서버소켓, 소켓, 각종 채널
  - java.lang.AutoCloseable 인터페이스 구현하고 있어야 함



FileInputSteam.java_AutoCloseable 구현 클래스

```java
public class FileInputStream implements AutoCloseable {
    private String file;

    public FileInputStream(String file) {
        this.file = file;
    }

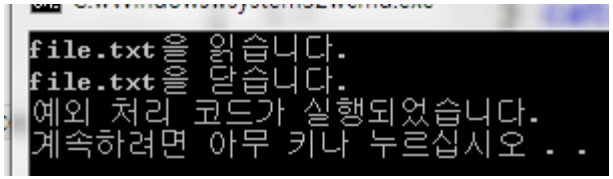
    public void read() {
        System.out.println(file + "을 읽습니다.");
    }

    @Override
    public void close() throws Exception {
        System.out.println(file + "을 닫습니다.");
    }
}
```



TryWithResourceExample.java_AutoCloseable 구현 클래스

```
1  public class TryWithResourceExample {
2      public static void main(String[] args) {
3          try (FileInputStream fis = new FileInputStream("file.txt")) {
4              fis.read();
5              throw new Exception();
6          } catch(Exception e) {
7              System.out.println("예외 처리 코드가 실행되었습니다.");
8          }
9      }
10 }
11
```



관련 클래스

FileInputStream.java
TryWithResourceExample.java

6절. 예외 떠 넘기기

❖ throws

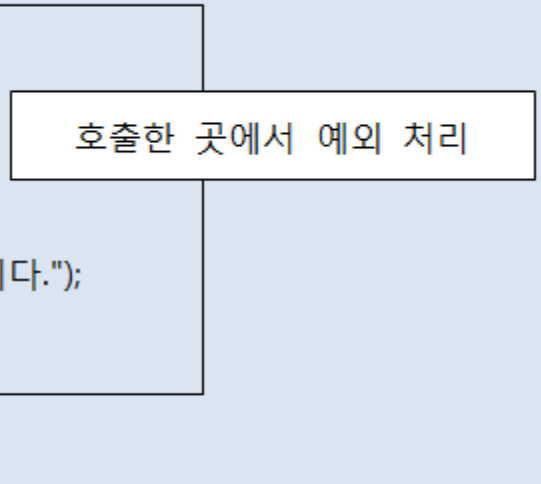
- **메소드 선언부 끝에 작성**

```
리턴타입 메소드명(매개변수,...) throws 예외클래스 1, 예외클래스 2, ... {
}
```

- **메소드에서 처리하지 않은 예외를 호출한 곳으로 떠 넘기는 역할**

```
public void method1() {
   try {
      method2();
   } catch(ClassNotFoundException e) {
      //예외 처리 코드
      System.out.println("클래스가 존재하지 않습니다.");
   }
}

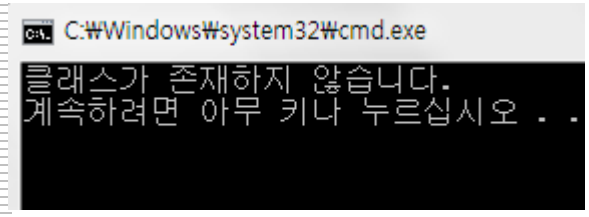
public void method2() throws ClassNotFoundException {
   Class clazz = Class.forName("java.lang.String2");
}
```



ThrowsExample.java_예외 처리 떠넘기기

```java
public class ThrowsExample {
    public static void main(String[] args) {
        try {
            findClass();
        } catch(ClassNotFoundException e) {
            System.out.println("클래스가 존재하지 않습니다.");
        }
    }

    public static void findClass() throws ClassNotFoundException {
        Class clazz = Class.forName("java.lang.String2");
    }
}
```



7절. 사용자 정의 예외와 예외 발생

- 사용자 정의 예외 클래스 선언
  - 자바 표준 API에서 제공하지 않는 예외
  - 애플리케이션 서비스와 관련된 예외
    - Ex) 잔고 부족 예외, 계좌 이체 실패 예외, 회원 가입 실패 예외….
  - 사용자 정의 예외 클래스 선언 방법

```
public class XXXException extends [ Exception | RuntimeException ] {
    public XXXException() { }
    public XXXException(String message) { super(message); }
}
```

```
public class BalanceInsufficientException extends Exception {
    public BalanceInsufficientException() { }
    public BalanceInsufficientException(String message) {
        super(message);
    }
```

7절. 사용자 정의 예외와 예외 발생

❖ 예외 발생 시키기

- 코드에서 예외 발생시키는 법

```
throw new XXXException()
throw new XXXException("메시지");
```

```
public void method() throws XXXException {
    throw new XXXException("메시지");
}
```

- 호출된 곳에서 발생한 예외를 처리하도록

```
public void withdraw(int money) throws BalanceInsufficientException {
    if(balance < money) {
        throw new BalanceInsufficientException("잔고부족:"+(money-balance)+" 모자람");
    }
```



8절. 예외 정보 얻기

❖ getMessage()

- 예외 발생시킬 때 생성자 매개값으로 사용한 메시지 리턴

```
throw new XXXException("예외  메시지");
```

- 원인 세분화하기 위해 예외 코드 포함(예: 데이터베이스 예외 코드)

- catch() 절에서 활용

```
} catch(Exception  e) {
    String message  = e.getMessage();
}
```

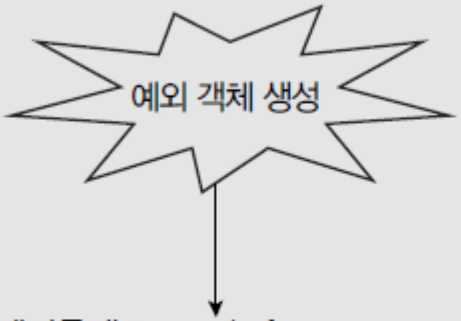


8절. 예외 정보 얻기

❖ printStackTrace()

- 예외 발생 코드 추적한 내용을 모두 콘솔에 출력
- 프로그램 테스트하면서 오류 찾을 때 유용하게 활용

```
try {
```




```
} catch(예외클래스    e) {
    //예외가 가지고 있는 Message 얻기
    String message = e.getMessage();

    //예외의 발생 경로를 추적
    e.printStackTrace();
}
```

BalanceInsufficientExample.java_사용자 정의 예외 클래스 1/3

```
1 public class BalanceInsufficientException extends Exception {
2     public BalanceInsufficientException() { }
3     public BalanceInsufficientException(String message) {
4         super(message);
5     }
6 }
```



Account.java_사용자 정의 예외 발생시키기 2/3

```
1  public class Account {
2      private long balance;
3
4      public Account() { }
5
6      public long getBalance() {
7          return balance;
8      }
9      public void deposit(int money) {
10         balance += money;
11     }
12     public void withdraw(int money) throws BalanceInsufficientException {
13         if(balance < money) {
14             throw new BalanceInsufficientException("잔고부족:"+(money-balance)+" 모자람");
15         }
16         balance -= money;
17     }
18 }
```



AccountExample.java_사용자 정의 예외 발생시키기 3/3

```java
public class AccountExample {
    public static void main(String[] args) {
        Account account = new Account();
        //예금하기
        account.deposit(10000);
        System.out.println("예금액: " + account.getBalance());
        //출금하기
        try {
            account.withdraw(30000);
        } catch(BalanceInsufficientException e) {
            String message = e.getMessage();
            System.out.println(message);
            System.out.println();
            e.printStackTrace();
        }
    }
}
```

```
C:₩Windows₩system32₩cmd.exe
예금액: 10000
잔고부족:20000 모자람

BalanceInsufficientException: 잔고부족:20000 모자람
        at Account.withdraw(Account.java:14)
        at AccountExample.main(AccountExample.java:9)
계속하려면 아무 키나 누르십시오 . . .
```

관련 클래스

- Account.java
- AccountExample.java
- BalanceInsufficientException.java

11장. 기본 API 클래스

이것이 자바다



Contents

- 1절. 자바 API 도큐먼트
- 2절. java.lang과 java.util 패키지
- 3절. Object 클래스
- 4절. Objects 클래스
- 5절. System 클래스
- 6절. Class 클래스
- 7절. String 클래스
- 8절. StringTokenizer 클래스



Contents

- 9절. StringBuffer, StringBuilder 클래스
- 10절. 정규 표현식과 Pattern 클래스
- 11절. Arrays 클래스
- 12절. 포장(Wrapper) 클래스
- 13절. Math, Random 클래스
- 14절. Date, Calendar 클래스
- 15절. Format 클래스
- 16절. java.time 패키지



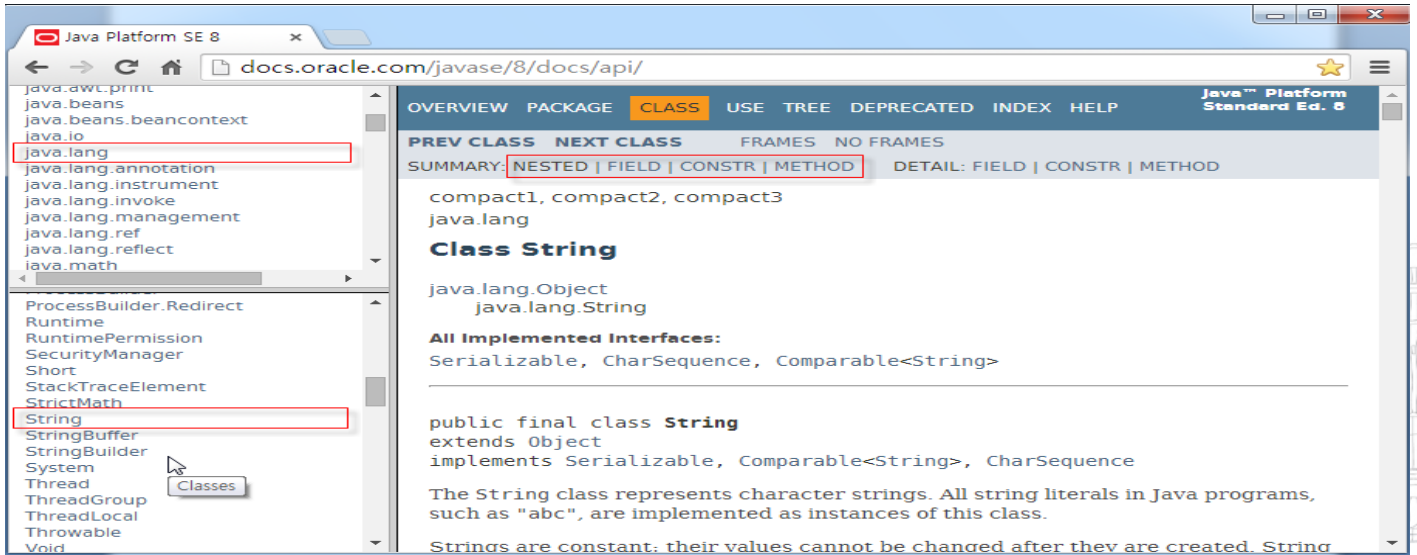
1절. 자바 API 도큐먼트

❖ 자바 API란?

- 자바에서 기본적으로 제공하는 라이브러리(library)
- 프로그램 개발에 자주 사용되는 클래스 및 인터페이스 모음

❖ API 도큐먼트 (p.454~456)

- 쉽게 API 찾아 이용할 수 있도록 문서화한 것
- HTML 페이지로 작성되어 있어 웹 브라우저로 바로 볼 수 있음
  - http://docs.oracle.com/javase/8/docs/api/



2절. java.lang과 java.util 패키지

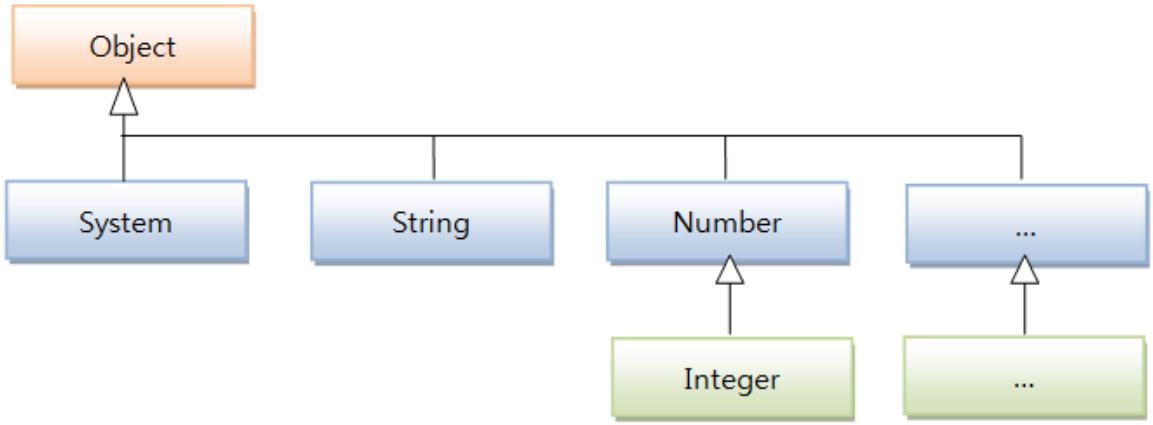
❖ java.lang 패키지

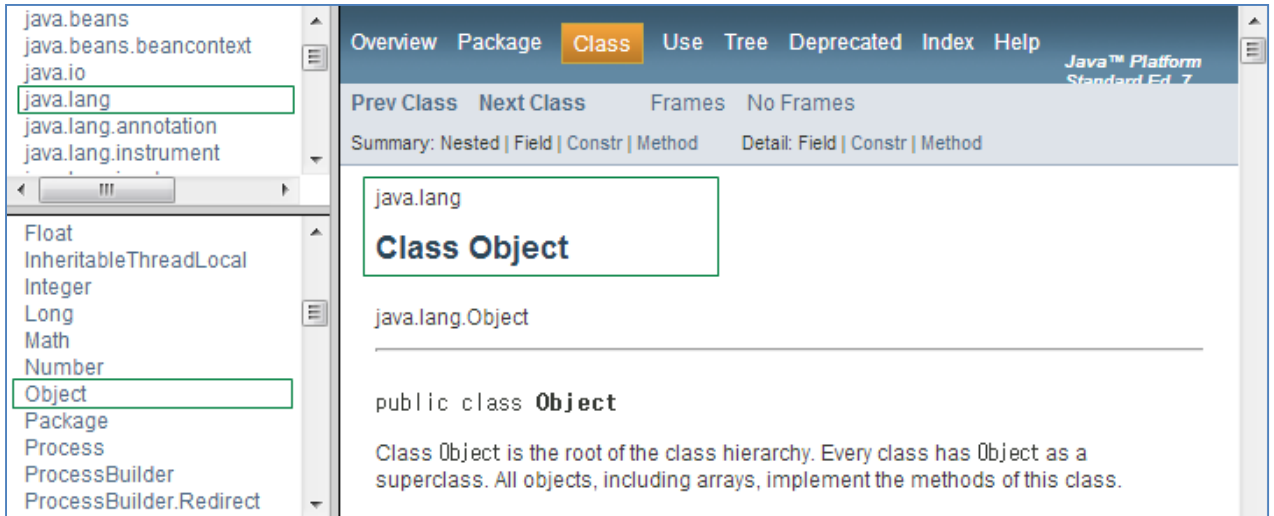
- 자바 프로그램의 기본적인 클래스를 담은 패키지
- 포함된 클래스와 인터페이스는 import 없이 사용
- 주요 클래스

클래스		용도
Object		- 자바 클래스의 최상위 클래스로 사용
System		- 표준 입력장치(키보드)로부터 데이터를 입력 받을 때 사용 - 표준 출력장치(모니터)로 출력하기 위해 사용 - 자바 가상 머신을 종료시킬 때 사용 - 쓰레기 수집기를 실행 요청할 때 사용
Class		- 클래스를 메모리로 로딩할 때 사용
String		- 문자열을 저장하고 여러가지 정보를 얻을 때 사용
StringBuffer, StringBuilder		- 문자열을 저장하고 내부 문자열을 조작할 때 사용
Math		- 수학 함수를 이용할 때 사용
Wrapper	Byte, Short, Character Integer, Float, Double Boolean	- 기본 타입의 데이터를 갖는 객체를 만들 때 사용 - 문자열을 기본 타입으로 변환할 때 사용 - 입력값 검사에 사용

3절. Object 클래스

- 자바의 최상위 부모 클래스
  - 다른 클래스 상속하지 않으면 java.lang.Object 클래스 상속 암시
  - Object의 메소드는 모든 클래스에서 사용 가능





3절. Object 클래스

❖ 객체 비교(equals() 메소드)

```
public boolean equals(Object obj) { ... }
```

- 기본적으로 == 연산자와 동일한 결과 리턴 (번지 비교)

```
Object obj1 = new Object();
Object obj2 = new Object();

boolean result = obj1.equals(obj2);
                 기준 객체    비교 객체

boolean result = (obj1 == obj2)
```

결과가 동일

- 논리적 동등 위해 오버라이딩 필요
  - 논리적 동등이란?
    - 같은 객체이건 다른 객체이건 상관없이 객체 저장 데이터 동일
  - Object의 equals() 메소드
    - 재정의하여 논리적 동등 비교할 때 이용



Member.java_객체 동등 비교(equals() 메소드) 1/2

```java
public class Member {
    public String id;

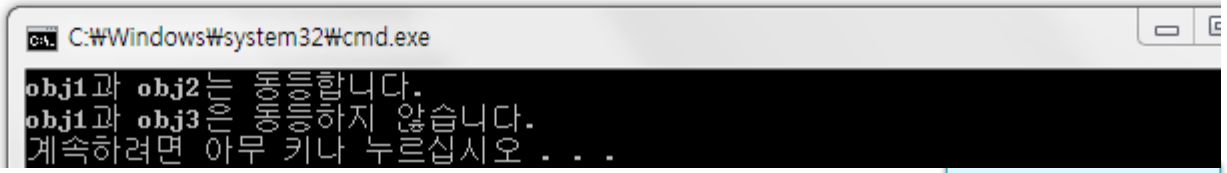
    public Member(String id) {
        this.id = id;
    }

    @Override
    public boolean equals(Object obj) {
        if(obj instanceof Member) {
            Member member = (Member) obj;
            if(id.equals(member.id)) {
                return true;
            }
        }
        return false;
    }
}
```



MemberExample.java_객체 동등 비교(equals() 메소드) 2/2

```
1  public class MemberExample {
2      public static void main(String[] args) {
3          Member obj1 = new Member("blue");
4          Member obj2 = new Member("blue");
5          Member obj3 = new Member("red");
6
7          if(obj1.equals(obj2)) {
8              System.out.println("obj1과 obj2는 동등합니다.");
9          } else {
10             System.out.println("obj1과 obj2는 동등하지 않습니다.");
11         }
12
13         if(obj1.equals(obj3)) {
14             System.out.println("obj1과 obj3은 동등합니다.");
15         } else {
16             System.out.println("obj1과 obj3은 동등하지 않습니다.");
17         }
18     }
19 }
20
```



관련 클래스

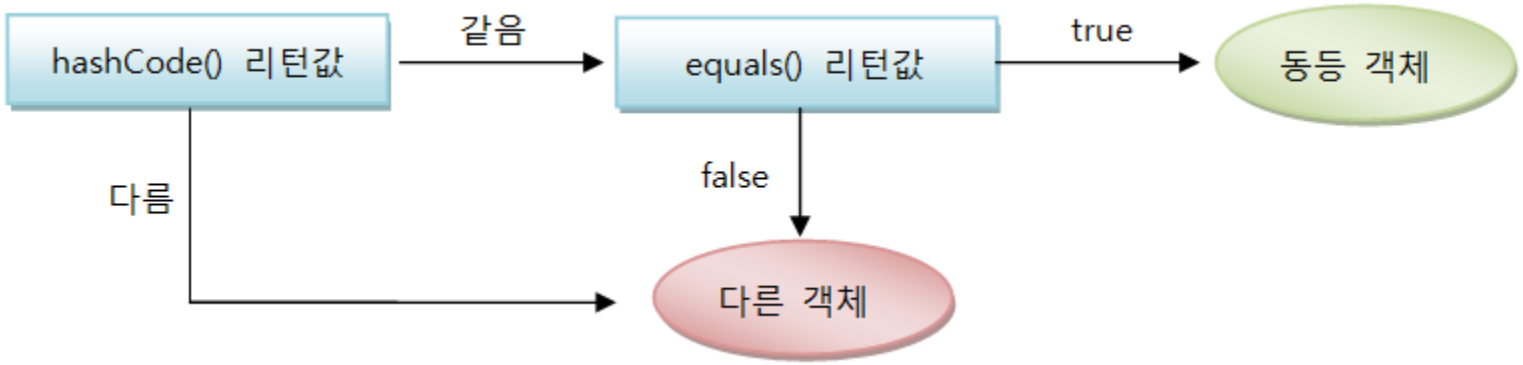
- Member.java
- MemberExample.java

3절. Object 클래스

❖ 객체 해시코드(hashCode()) (p.461~463)

- 객체 해시코드란?
  - 객체 식별할 하나의 정수값
  - 객체의 메모리 번지 이용해 해시코드 만들어 리턴
    - 개별 객체는 해시코드가 모두 다름

- 논리적 동등 비교 시 hashCode() 오버라이딩의 필요성
  - 컬렉션 프레임워크의 HashSet, HashMap, Hashtable 과 같은 클래스는 두 객체가 동등한 객체인지 판단할 때 아래와 같은 과정을 거침



Key.java_hash() 메소드를 재정의하지 않음)

```java
public class Key {
    public int number;

    public Key(int number) {
        this.number = number;
    }

    @Override
    public boolean equals(Object obj) {
        if(obj instanceof Key) {
            Key compareKey = (Key) obj;
            if(this.number == compareKey.number) {
                return true;
            }
        }
        return false;
    }


}
```



KeyExample.java_hash() 다른키로 인식)

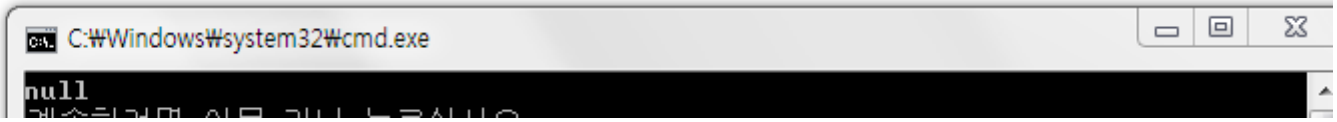
```java
import java.util.HashMap;

public class KeyExample {
    public static void main(String[] args) {
        //Key 객체를 식별키로 사용해서 String 값을 저장하는 HashMap 객체 생성
        HashMap<Key, String> hashMap = new HashMap<Key, String>();

        //식별키 "new Key(1)" 로 "홍길동"을 저장함
        hashMap.put(new Key(1), "홍길동");

        //식별키 "new Key(1)" 로 "홍길동"을 읽어옴
        String value  = hashMap.get(new Key(1));
        System.out.println(value);

        }
}
```



관련 클래스

Key.java
KeyExample.java

Key.java_hashcode() 메소드 재정의 추가

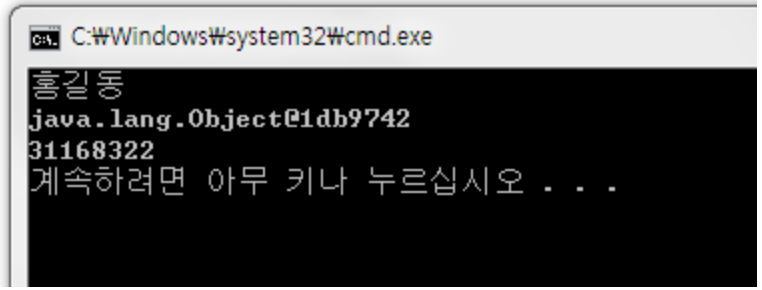
```
1  public class Key {
2      public int number;
3
4      public Key(int number) {
5          this.number = number;
6      }
7
8      @Override
9      public boolean equals(Object obj) {
10         if(obj instanceof Key) {
11             Key compareKey = (Key) obj;
12             if(this.number == compareKey.number) {
13                 return true;
14             }
15         }
16         return false;
17     }
18
19     @Override
20     public int hashCode() {
21         return number;
22     }
23 }
```

추가

KeyExample.java_hashCode() 메소드를 재정의추가)

```
1  import java.util.HashMap;
2
3  public class KeyExample {
4      public static void main(String[] args) {
5          //Key 객체를 식별키로 사용해서 String 값을 저장하는 HashMap 객체 생성
6          HashMap<Key, String> hashMap = new HashMap<Key, String>();
7
8          //식별키 "new Key(1)" 로 "홍길동"을 저장함
9          hashMap.put(new Key(1), "홍길동");
10
11         //식별키 "new Key(1)" 로 "홍길동"을 읽어옴
12         String value  = hashMap.get(new Key(1));
13         System.out.println(value);
14
15         Object obj = new Object();
16         System.out.println(obj);
17         System.out.println(obj.hashCode());
18     }
19 }
```

추가



관련 클래스

Key.java
KeyExample.java

3절. Object 클래스

❖ 객체 문자정보(toString())

- 객체를 문자열로 표현한 값
- Object 클래스의toString() 메소드는 객체의 문자 정보 리턴

```
Object obj = new Object();
System.out.println( obj.toString() );

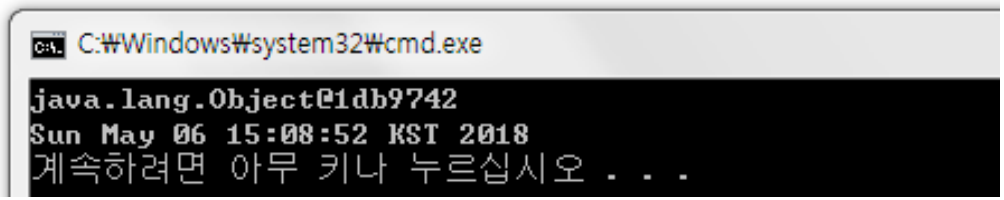
[실행 결과]
java.lang.Object@de6ced
```

- 일반적으로 의미 있는 문자정보가 나오도록 재정의
  - Date 클래스- 현재 시스템의 날짜와 시간 정보 리턴
  - String 클래스 - 저장하고 있는 문자열 리턴
- System.out.pritnln(Object) 메소드
  - Object의 toString()의 리턴값 출력



TostringExample.java_객체의 문자 정보(tostring() 메소드)

```
1  import java.util.Date;
2
3  public class ToStringExample {
4      public static void main(String[] args) {
5          Object obj1 = new Object();
6          Date obj2 = new Date();
7          System.out.println(obj1.toString());
8          System.out.println(obj2.toString());
9      }
10 }
11
12
```



SmartPhone.java_객체의 문자 정보(tostring() 메소드)

```java
public class SmartPhone {
    private String company;
    private String os;

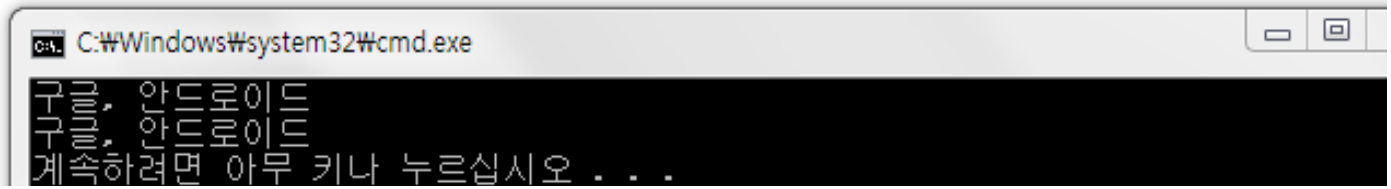
    public SmartPhone(String company, String os) {
        this.company = company;
        this.os = os;
    }

    @Override
    public String toString() {
        return company + ", " + os;
    }
}
```



SmartPhoneExample.java_객체의 문자 정보(tostring() 메소드)

```
1  public class SmartPhoneExample {
2      public static void main(String[] args) {
3          SmartPhone myPhone = new SmartPhone("구글", "안드로이드");
4
5          String strObj = myPhone.toString();
6          System.out.println(strObj);
7
8          System.out.println(myPhone);
9      }
10 }
11
```



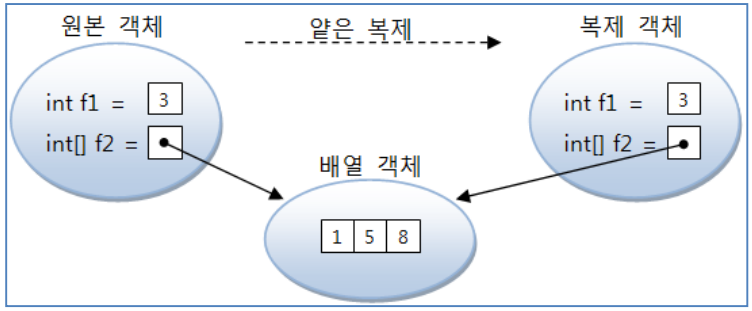
관련 클래스

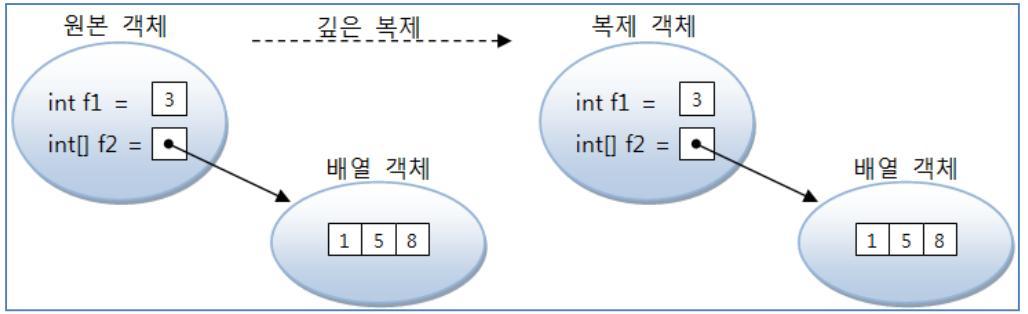
- SmartPhone.java
- SmartPhoneExample.java
- ToStringExample.java

3절. Object 클래스

❖ 객체 복제(clone())

- 원본 객체의 필드 값과 동일한 값을 가지는 새로운 객체 생성하는 것
- 복제 종류
  - 얕은 복제(thin clone): 필드 값만 복제 (참조 타입 필드는 번지 공유)
  - 깊은 복제(deep clone): 참조하고 있는 객체도 복제





- Object의 clone() 메소드
  - 동일한 필드 값 을 가진 얕은 복제된 객체 리턴
  - java.lang.Cloneable 인터페이스 구현한 객체만 복제 가능

- 깊은 복제 - clone() 메소드 재정의하고 참조 객체도 복제해야

Member.java_복제할 수 있는 클래스 선언_얕은 복제

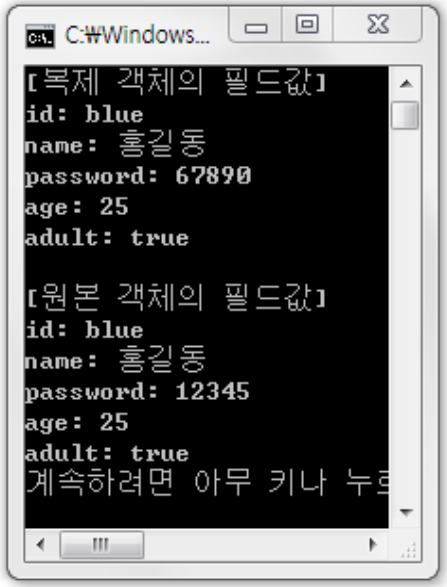
```java
public class Member implements Cloneable {
    public String id;
    public String name;
    public String password;
    public int age;
    public boolean adult;

    public Member(String id,  String name, String password, int age, boolean adult ) {
        this.id = id;
        this.name = name;
        this.password = password;
        this.age = age;
        this.adult = adult;
    }

    public Member getMember() {
        Member cloned = null;
        try {
            cloned = (Member) clone();
        } catch (CloneNotSupportedException e) {
            e.printStackTrace();
        }
        return cloned;
    }
}
```

MemberExample.java_복제 객체를 변경하더라도 원본 객체는 변함 없음

```
1  public class MemberExample {
2      public static void main(String[] args) {
3          //원본 객체 생성
4          Member original = new Member("blue", "홍길동", "12345", 25, true);
5
6          //복제 객체를 얻은 후에 패스워드 변경
7          Member cloned = original.getMember();
8          cloned.password = "67890";
9
10         System.out.println("[복제 객체의 필드값]");
11         System.out.println("id: " + cloned.id);
12         System.out.println("name: " + cloned.name);
13         System.out.println("password: " + cloned.password);
14         System.out.println("age: " + cloned.age);
15         System.out.println("adult: " + cloned.adult);
16
17         System.out.println();
18
19         System.out.println("[원본 객체의 필드값]");
20         System.out.println("id: " + original.id);
21         System.out.println("name: " + original.name);
22         System.out.println("password: " + original.password);
23         System.out.println("age: " + original.age);
24         System.out.println("adult: " + original.adult);
25     }
26 }
27
```



관련 클래스

Member.java
MemberExample.java

Member.java_clone()을 재정의해서 _깊은 복제로 변경 1/2

```java
import java.util.Arrays;

public class Member implements Cloneable {
    public String name;
    public int age;
    public int[] scores;
    public Car car;

    public Member(String name, int age, int[] scores, Car car) {
        this.name = name;
        this.age = age;
        this.scores = scores;
        this.car = car;
    }

    @Override
    protected Object clone() throws CloneNotSupportedException {
        //먼저 얕은 복사를 해서 name, age를 복제한다.
        Member cloned = (Member) super.clone();
        //scores를 복제한다.
        cloned.scores = Arrays.copyOf(this.scores, this.scores.length);
        //car를 복제한다.
        cloned.car = new Car(this.car.model);
        //깊은 복제된 Member 객체를 리턴
        return cloned;
    }

    public Member getMember() {
        Member cloned = null;
        try {
            cloned = (Member) clone();
        } catch (CloneNotSupportedException e) {
            e.printStackTrace();
        }
        return cloned;
    }
}
```



car.java_Car 클래스

```java
public class Car {
    public String model;

    public Car(String model) {
        this.model = model;
    }
}
```



MemberExample.java_깊은 복제 후 복제본 변경은 원본에 영향을 미치지 않는다.

```java
public class MemberExample {
    public static void main(String[] args) {
        //원본 객체 생성
        Member original = new Member("홍길동", 25, new int[] {90, 90}, new Car("소나타"));

        //복제 객체를 얻은 후에 참조 객체의 값을 변경
        Member cloned = original.getMember();
        cloned.scores[0] = 100;
        cloned.car.model = "그랜저";

        System.out.println("[복제 객체의 필드값]");
        System.out.println("name: " + cloned.name);
        System.out.println("age: " + cloned.age);
        System.out.print("scores: {");
        for(int i=0; i<cloned.scores.length; i++) {
            System.out.print(cloned.scores[i]);
            System.out.print((i==(cloned.scores.length-1))?"":",");
        }
        System.out.println("}");
        System.out.println("car: " + cloned.car.model);

        System.out.println();
           System.out.println("[원본 객체의 필드값]");
           System.out.println("name: " + original.name);
           System.out.println("age: " + original.age);
           System.out.print("scores: {");
           for(int i=0; i<original.scores.length; i++) {
               System.out.print(original.scores[i]);
               System.out.print((i==(original.scores.length-1))?"":",");
           }
           System.out.println("}");
           System.out.println("car: " + original.car.model);
    }
}
```

관련 클래스

- Car.java
- Member.java
- MemberExample.java

3절. Object 클래스

❖ 객체 소멸자(finalize())

- GC는 객체를 소멸하기 직전 객체 소멸자(finalize()) 실행
- Object의 finalize() 는 기본적으로 실행 내용이 없음
- 객체가 소멸되기 전에 실행할 코드가 있다면?
  - Object의 finalize() 재정의

```
@Override
protected void finalize() throws Throwable {
   System.out.println(no + "번 객체의 finalize()가 실행됨");
}
```

- 될 수 있으면 소멸자는 사용하지 말 것
  - GC는 메모리의 모든 쓰레기 객체를 소멸하지 않음
  - GC의 구동 시점이 일정하지 않음



Counter.java_finalize()메소드 재정의

```java
public class Counter {
    private int no;

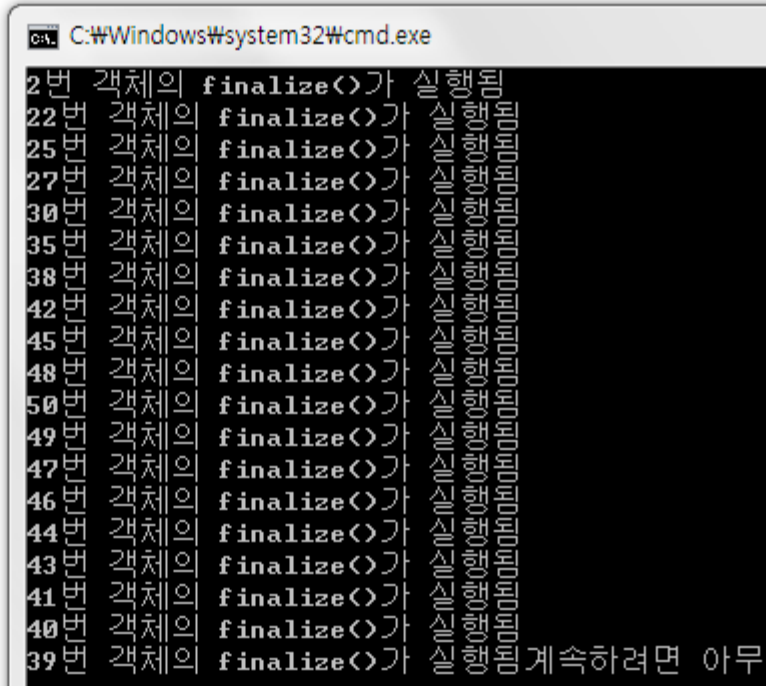
    public Counter(int no) {
        this.no = no;
    }

    @Override
    protected void finalize() throws Throwable {
        System.out.println(no + "번 객체의 finalize()가 실행됨");
    }
}
```



FinalizeExample.java_finalize()메소드 실행 확인

```
1  public class FinalizeExample {
2      public static void main(String[] args) {
3          Counter counter = null;
4          for(int i=1; i<=50; i++) {
5              counter = new Counter(i);
6              counter = null;
7              System.gc();
8          }
9      }
10 }
11
```



관련 클래스

Counter.java
FinalizeExample.java

4절. Objects 클래스

❖ Objects 클래스

- **Object의 유틸리티 클래스**

리턴타입	메소드(매개변수)	설명
int	compare(T a, T b, Comparator<T> c)	두 객체 a 와 b 를 Comparator 를 사용해서 비교
boolean	deepEquals(Object a, Object b)	두 객체의 깊은 비교(필드도 비교)
boolean	equals(Object a, Object b)	두 객체의 얕은 비교(번지만 비교)
int	hash(Object... values)	매개값이 저장된 배열의 해시코드 생성
int	hashCode(Object o)	객체의 해시코드 생성
boolean	isNull(Object obj)	객체가 널 인지 조사
boolean	nonNull(Object obj)	객체가 널이 아닌지 조사
T	requireNonNull(T obj)	객체가 널인 경우 예외 발생
T	requireNonNull(T obj, String message)	객체가 널인 경우 예외 발생(주어진 예외 메시지 포함)
T	requireNonNull(T obj, Supplier<String> messageSupplier)	객체가 널인 경우 예외 발생(람다식이 만든 예외 메시지 포함)
String	toString(Object o)	객체의 toString() 리턴값 리턴
String	toString(Object o, String nullDefault)	객체의 toString() 리턴값 리턴, 첫번째 매개값이 null 일 경우 두번째 매개값 리턴

4절. Objects 클래스

❖ 객체 비교 (Objects.compare(T a, T b, Comparator<T> c))

- a, b 두 객체를 비교자(c)로 비교해 int값 리턴

- Comparator<T> 인터페이스
  - 제너릭 인터페이스 타입
  - T 타입의 객체를 비교하는 compare(T a, T b) 메소드 가짐

```
public interface Comparator<T> {
   int compare(T  a, T b)
}
```

– 비교 예제 p.474~476



CompareExample.java_비교자

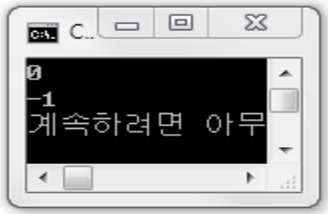
```java
import java.util.Comparator;
import java.util.Objects;

public class CompareExample {
    public static void main(String[] args) {
        Student s1 = new Student(1);
        Student s2 = new Student(1);
        Student s3 = new Student(2);

        int result = Objects.compare(s1, s2, new  StudentComparator());
        System.out.println(result);
        result = Objects.compare(s1, s3, new  StudentComparator());
        System.out.println(result);
    }

    static class Student {
        int sno;
        Student(int sno) {
            this.sno = sno;
        }
    }

    static class StudentComparator implements Comparator<Student> {
        @Override
        public int compare(Student a, Student b) {
            /*if(a.sno<b.sno) return -1;
            else if(a.sno == b.sno) return 0;
            else return 1;*/
            return Integer.compare(a.sno, b.sno);
        }
    }
}
```



4절. Objects 클래스

❖ 동등 비교(equals()와 deepEquals())

- 두 객체의 동등 비교
- Objects.equals(Object a, Object b)

a	b	Objects.equals(a, b)
not null	not null	a.equals(b)
null	not null	false
not null	null	false
null	null	true

- deepEquals(Object a, Object b)
  - 비교할 객체가 배열일 경우 항목 값까지도 비교

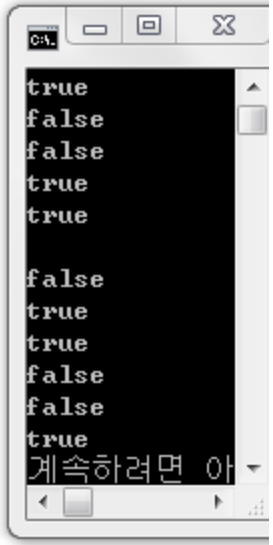
a	b	Objects.deepEquals(a, b)
not null (not array)	not null (not array)	a.equals(b)
not null (array)	not null (array)	Arrays.deepEquals(a, b)
not null	null	false
null	not null	false
null	null	true

EqualsAndDeepEqualsExample.java_객체 동등 비교

```java
import java.util.Arrays;
import java.util.Objects;

public class EqualsAndDeepEqualsExample {
    public static void main(String[] args) {
        Integer o1 = 1000;
        Integer o2 = 1000;
        System.out.println(Objects.equals(o1, o2));
        System.out.println(Objects.equals(o1, null));
        System.out.println(Objects.equals(null, o2));
        System.out.println(Objects.equals(null, null));
        System.out.println(Objects.deepEquals(o1, o2) + "\n");

        Integer[] arr1 = { 1, 2 };
        Integer[] arr2 = { 1, 2 };
        System.out.println(Objects.equals(arr1, arr2));
        System.out.println(Objects.deepEquals(arr1, arr2));
        System.out.println(Arrays.deepEquals(arr1, arr2));
        System.out.println(Objects.deepEquals(null, arr2));
        System.out.println(Objects.deepEquals(arr1, null));
        System.out.println(Objects.deepEquals(null, null));
    }
}
```



4절. Objects 클래스

❖ 해시코드 생성(hash(), hashCode())

- Objects.hash(Object… values)
  - 매개값으로 주어진 값들 이용해 해시 코드 생성하는 역할
  - Arrays.hashCode(Object[]) 호출해 해시코드 얻어 리턴
  - 클래스의 hashCode()의 리턴값 생성할 때 유용하게 사용

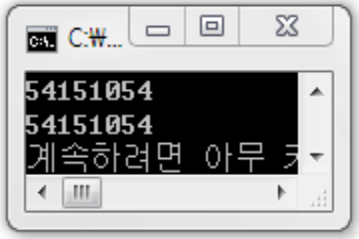
```
@Override
public int hashCode() {
    return Objects.hash(field1, field2, field3);
}
```

- Objects.hashCode(Object o)
  - o.hashCode() 호출하고 받은 값 리턴
  - 매개값이 null 이면 0 리턴



HashcodeExample.java_해시코드 생성

```
1  import java.util.Objects;
2
3  public class HashCodeExample {
4      public static void main(String[] args) {
5          Student s1 = new Student(1, "홍길동");
6          Student s2 = new Student(1, "홍길동");
7          System.out.println(s1.hashCode());
8          System.out.println(Objects.hashCode(s2));
9      }
10
11     static class Student {
12         int sno;
13         String name;
14         Student(int sno, String name) {
15             this.sno = sno;
16             this.name = name;
17         }
18         @Override
19         public int hashCode() {
20             return Objects.hash(sno, name);
21         }
22     }
23 }
24
```



4절. Objects 클래스

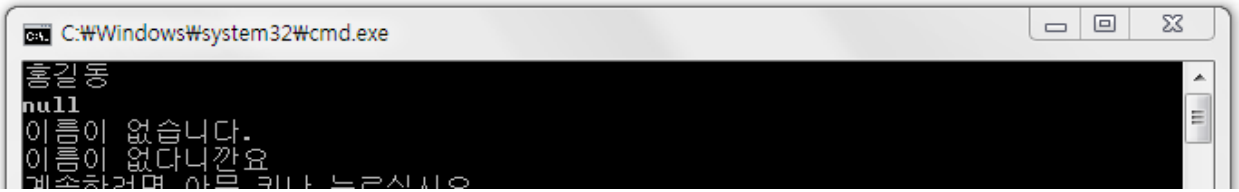
❖ 널 여부 조사(isNull(), nonNull(), requireNonNull())

- Objects.isNull(Object obj)
  - obj가 null일 경우 true
- Objects.nonNull(Object obj)
  - obj가 not null일 경우 true
- requireNonNull()

리턴타입	메소드(매개변수)	설명
T	requireNonNull(T obj)	not null → obj null → NullPointerException
T	requireNonNull(T obj, String message)	not null → obj null → NullPointerException(message)
T	requireNonNull(T obj, Supplier<String> msgSupplier)	not null → obj null → NullPointerException(msgSupplier.get())

NullExample.java_널(null) 여부 조사

```
1  import java.util.Objects;
2
3  public class NullExample {
4      public static void main(String[] args) {
5          String str1 = "홍길동";
6          String str2 = null;
7
8          System.out.println(Objects.requireNonNull(str1));
9
10         try {
11             String name = Objects.requireNonNull(str2);
12         } catch(Exception e) {
13             System.out.println(e.getMessage());
14         }
15
16         try {
17             String name = Objects.requireNonNull(str2, "이름이 없습니다.");
18         } catch(Exception e) {
19             System.out.println(e.getMessage());
20         }
21
22         try {
23             String name = Objects.requireNonNull(str2, ()->"이름이 없다니깐요");
24         } catch(Exception e) {
25             System.out.println(e.getMessage());
26         }
27     }
28 }
29
```



4절. Objects 클래스

❖ 객체 문자정보(toString())

- 객체의 문자정보 리턴

- 첫 번째 매개값이 not null – toString ( )으로 얻은 값을 리턴
- null이면 “null” 또는 두 번째 매개값인 nullDefault 리턴

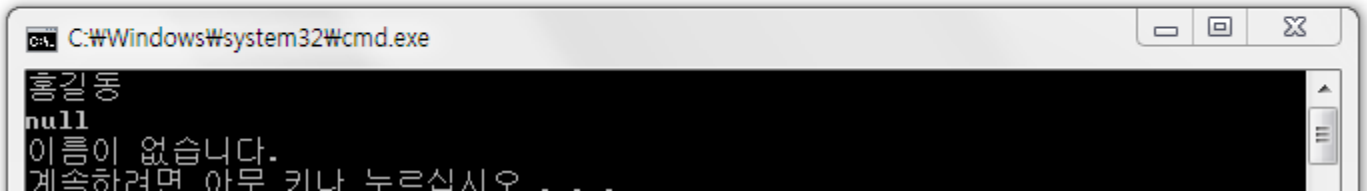


TostringExample.java_객체 문자 정보

```java
import java.util.Objects;

public class ToStringExample {
    public static void main(String[] args) {
        String str1 = "홍길동";
        String str2 = null;

        System.out.println(Objects.toString(str1));
        System.out.println(Objects.toString(str2));
        System.out.println(Objects.toString(str2, "이름이 없습니다."));
    }
}
```



5절. System 클래스

❖ System 클래스 용도

- 운영체제의 기능 일부 이용 가능
  - 프로그램 종료, 키보드로부터 입력, 모니터 출력, 메모리 정리, 현재 시간 읽기
  - 시스템 프로퍼티 읽기, 환경 변수 읽기

❖ 프로그램 종료(exit())

- 기능 - 강제적으로 JVM 종료

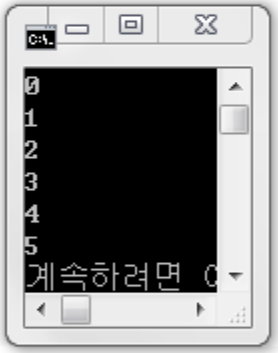
```
System.exit(0);
```

  - int 매개값을 지정하도록 - 종료 상태 값
    - 정상 종료일 경우 0, 비정상 종료일 경우 0 이외 다른 값
    - 어떤 값 주더라도 종료
  - 만약 특정 상태 값이 입력되었을 경우에만 종료하고 싶다면?
    - 자바의 보안 관리자 설정



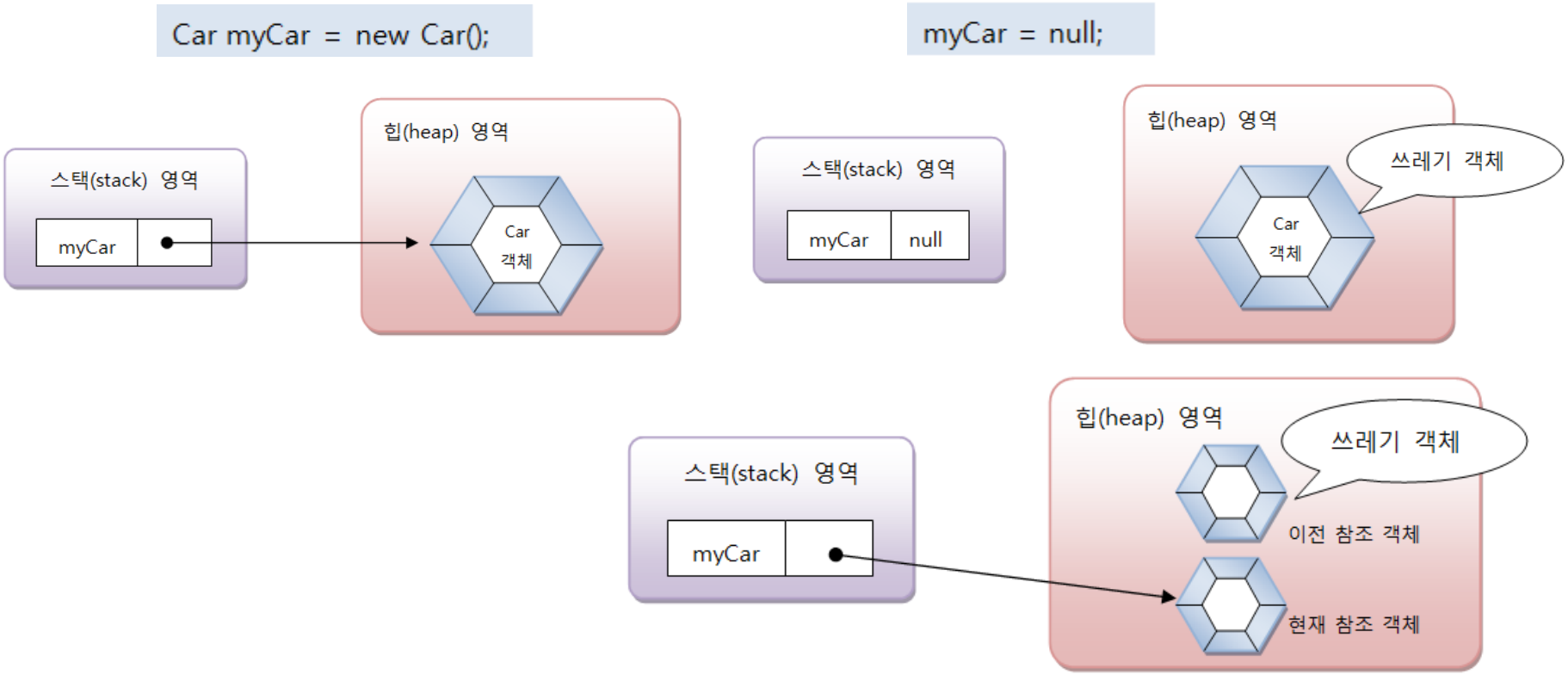
ExitExample.java_exit 메소드

```
1  public class ExitExample {
2      public static void main(String[] args)  {
3          //보안 관리자 설정
4          System.setSecurityManager(new SecurityManager(){
5              @Override
6              public void checkExit(int status) {
7                  if(status != 5) {
8                      throw new SecurityException();
9                  }
10             }
11         });
12
13         for(int i=0; i<10; i++) {
14             //i값 출력
15             System.out.println(i);
16             try {
17                 //JVM 종료 요청
18                 System.exit(i);
19             } catch(SecurityException e) { }
20         }
21     }
22 }
23
```



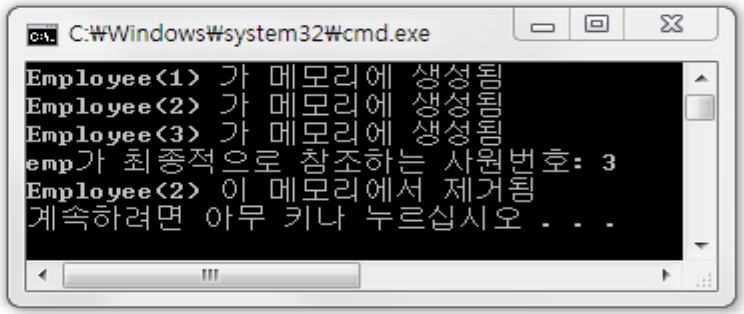
5절. System 클래스

❖ 쓰레기 수집기 실행 요청(gc()) (p.482~484)



GcExample.java_gc 메소드

```
1  public class GcExample {
2      public static void main(String[] args) {
3          Employee emp;
4
5          emp = new Employee(1);
6          emp = null;
7          emp = new Employee(2);
8          emp = new Employee(3);
9
10         System.out.print("emp가 최종적으로 참조하는 사원번호: ");
11         System.out.println(emp.eno);
12         System.gc();
13     }
14 }
15
16
17  class Employee {
18     public int eno;
19
20     public Employee(int eno) {
21         this.eno = eno;
22         System.out.println("Employee(" + eno + ") 가 메모리에 생성됨");
23     }
24
25     public void finalize() {
26         System.out.println("Employee(" + eno + ") 이 메모리에서 제거됨");
27     }
28 }
29
```



5절. System 클래스

❖ 현재 시각 읽기

- 현재 시간을 읽어 밀리 세컨드(currentTimeMillis() -> 1/1000초)와 나노세컨드(nanoTime()->1/10^9초) 단위의 long값 리턴

```
long time = System.currentTimeMillis();
long time = System.nanoTime();
```

- 주로 프로그램 실행 소요 시간 구할 때 이용



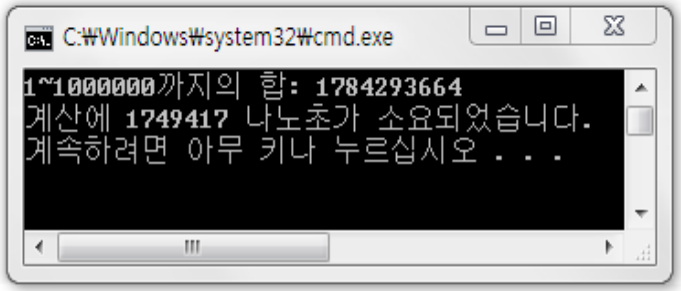
SystemTimeExample.java_프로그램 실행 소요 시간 구하기

```java
public class SystemTimeExample {
    public static void main(String[] args) {
        long time1 = System.nanoTime();

        int sum = 0;
        for(int i=1; i<=1000000; i++) {
            sum += i;
        }

        long time2 = System.nanoTime();

        System.out.println("1~1000000까지의 합: " + sum);
        System.out.println("계산에 " + (time2-time1) + " 나노초가 소요되었습니다.");
    }
}
```



5절. System 클래스

❖ 시스템 프로퍼티 읽기(getProperty())

- **시스템 프로퍼티란?**
  - JVM이 시작할 때 자동 설정되는 시스템의 속성값

- **대표적인 키와 값**

키(key)	설명	값(value)
java.version	자바의 버전	1.7.0_25
java.home	사용하는 JRE 의 파일 경로	<jdk 설치경로>₩jre
os.name	Operating system name	Windows 7
file.separator	File separator ("/" on UNIX)	₩
user.name	사용자의 이름	사용자계정
user.home	사용자의 홈 디렉토리	C:₩Users₩사용자계정
user.dir	사용자가 현재 작업 중인 디렉토리 경로	다양

- **시스템 프로퍼티 읽어오는 법**

```
String value = System.getProperty(String key);
```

getpropertyExample.java_시스템 프로퍼티 읽기

```java
import java.util.Properties;
import java.util.Set;

public class GetPropertyExample {
    public static void main(String[] args) {
        String osName = System.getProperty("os.name");
        String userName = System.getProperty("user.name");
        String userHome = System.getProperty("user.home");

        System.out.println("운영체제 이름: " + osName);
        System.out.println("사용자 이름: " + userName);
        System.out.println("사용자 홈디렉토리: " + userHome);

        System.out.println("----------------------------------");
        System.out.println(" [ key ]  value");
        System.out.println("----------------------------------");
        Properties props = System.getProperties();
        Set keys = props.keySet();
        for(Object objKey : keys) {
            String key = (String) objKey;
            String value = System.getProperty(key);
            System.out.println("[ " + key + " ]  " + value);
        }

    }
}
```

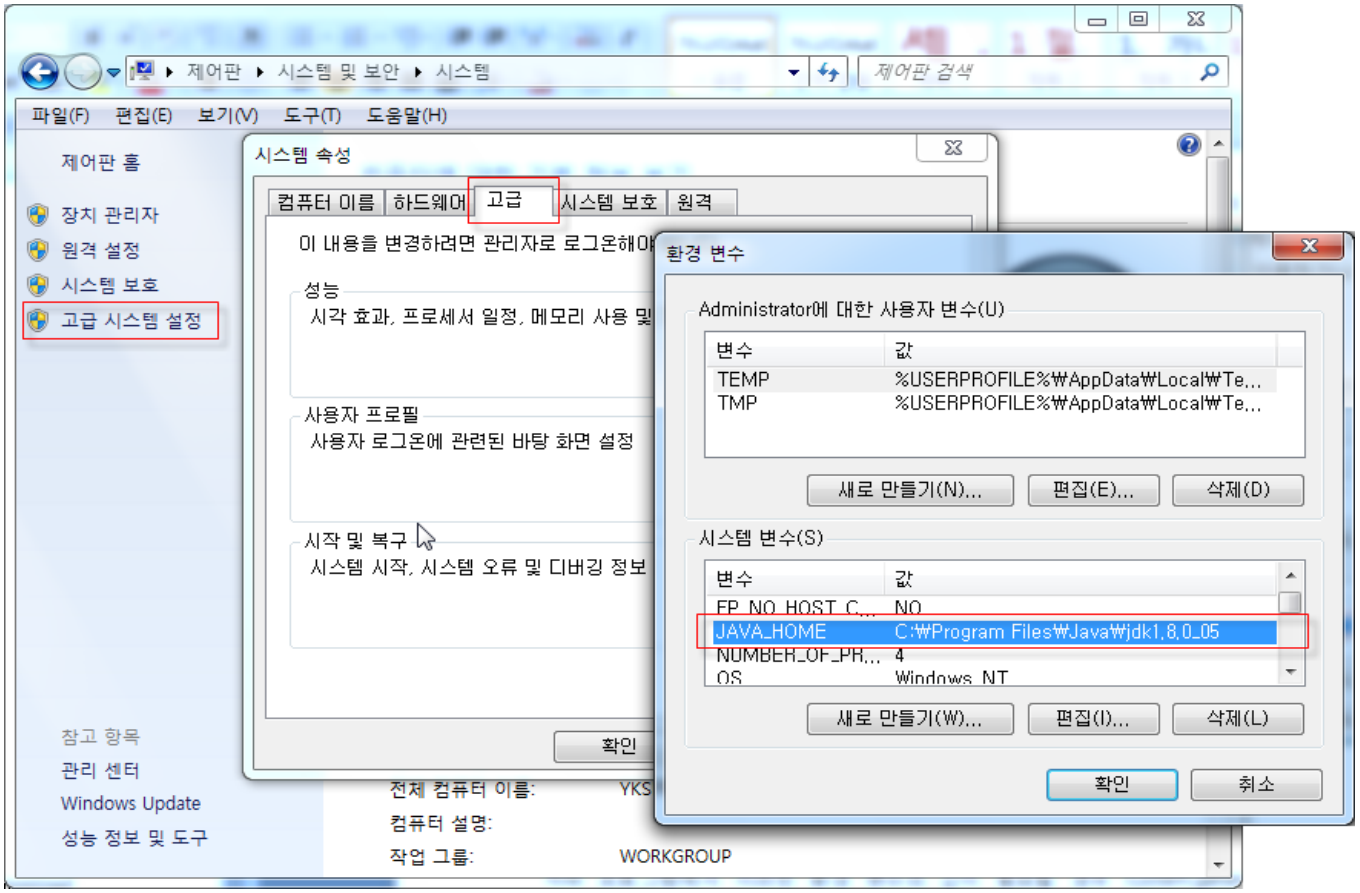
```
C:₩Windows₩system32₩cmd.exe
운영체제 이름: Windows 7
사용자 이름: user
사용자 홈디렉토리: C:₩Users₩user
---------------------------------
 [ key ]  value
---------------------------------
[ java.runtime.name ]  Java(TM) SE Runtime Environment
[ sun.boot.library.path ]  C:₩jdk1.8.0_151₩jre₩bin
```

5절. System 클래스

❖ 환경 변수 읽기(getenv())

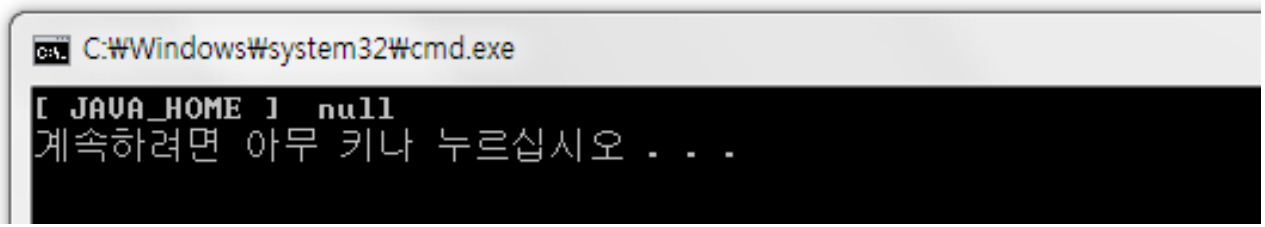
- 운영체제가 제공하는 환경 변수 값 (문자열) 을 읽음

```
String value = System.getenv (String name);
```



SystemEnvExample.java_JAVA_HOME 환경 변수 값 읽기

```
1 public class SystemEnvExample {
2     public static void main(String[] args) {
3         String javaHome = System.getenv("JAVA_HOME");
4         System.out.println("[ JAVA_HOME ]  " + javaHome);
5     }
6 }
7
```



6절. Class 클래스

❖ Class 클래스

- 클래스와 인터페이스의 메타 데이터 관리
  - 메타데이터: 클래스의 이름, 생성자 정보, 필드 정보, 메소드 정보

❖ Class 객체 얻기(getClass(), forName())

- 객체로부터 얻는 방법

```
Class clazz = obj.getClass();
```

- 문자열로부터 얻는 방법

```
try {
   Class clazz = Class.forName(String  className);
} catch (ClassNotFoundException  e) {
}
```

❖ 리플렉션 (p.491~493)

- 클래스의 생성자, 필드, 메소드 정보를 알아내는 것

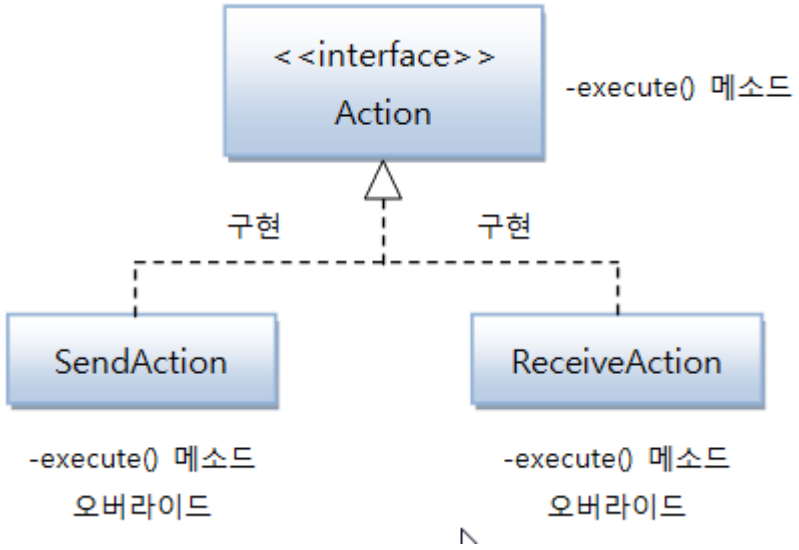


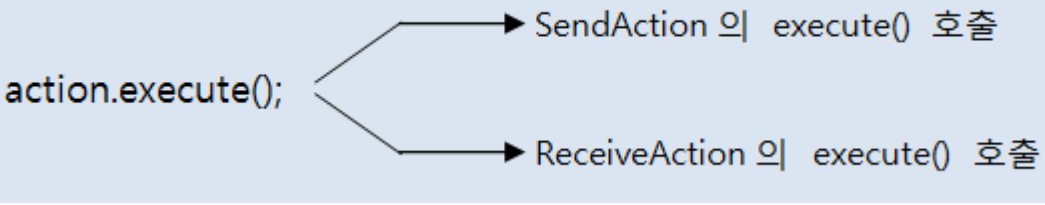
6절. Class 클래스

❖ 동적 객체 생성(newInstance()) (p.493~496)

- 실행 도중 클래스 이름이 결정될 경우 동적 객체 생성 가능

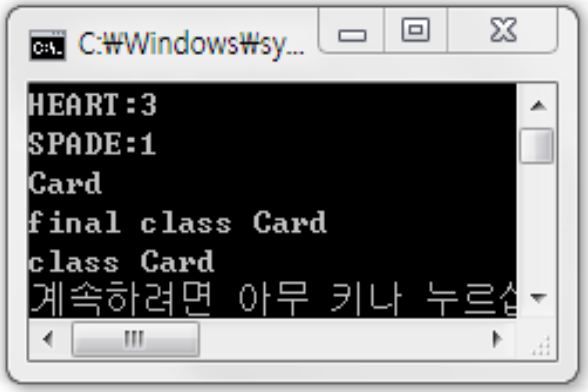
```
Class clazz = Class.forName("SendAction" 또는 "ReceiveAction");
Action action = (Action) clazz.newInstance();
action.execute();
```





Classs객체를 이용해서 동적으로 객체를 생성하는 예제

```
1  final class Card {
2      String kind;
3      int num;
4
5      Card() {
6          this("SPADE", 1);
7      }
8
9      Card(String kind, int num) {
10         this.kind = kind;
11         this.num  = num;
12     }
13
14     public String toString() {
15         return kind + ":" + num;
16     }
17 }
18
19 class ClassEx1 {
20     public static void main(String[] args) throws Exception {
21         Card c  = new Card("HEART", 3);       // new연산자로 객체 생성
22         Card c2 = Card.class.newInstance();   // Class객체를 통해서 객체 생성
23
24         Class cObj = c.getClass();
25
26         System.out.println(c);
27         System.out.println(c2);
28         System.out.println(cObj.getName());
29         System.out.println(cObj.toGenericString());
30         System.out.println(cObj.toString());
31     }
32 }
33
```



7절. String 클래스

❖ 생성자

▪ byte[] 배열을 문자열로 변환하는 생성자

```
//배열 전체를 String 객체 생성
String str = new String(byte[] bytes);
//지정한 문자셋으로 디코딩
String str = new String(byte[] bytes, String charsetName);

//배열의 offset 인덱스 위치부터 length 개 만큼 String 객체 생성
String str = new String(byte[] bytes, int offset, int length);
//지정한 문자셋으로 디코딩
String str = new String(byte[] bytes, int offset, int length, String charsetName)
```

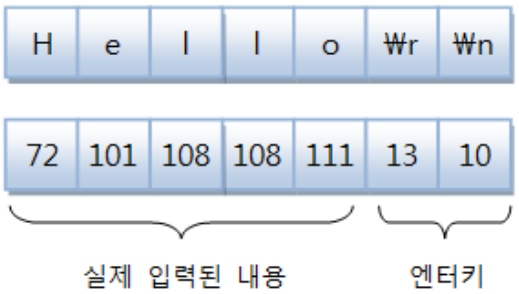
파일의 내용을 읽거나,
네트워크를 통해 받은 데이터는
보통 byte[] 배열이므로
이것을 문자열로 변환하기 위해 사용

▪ 키보드로부터 읽은 바이트 배열을 문자열로 변환

```
byte[] bytes = new byte[100];
int readByteNo = System.in.read(bytes);
String str = new String(bytes, 0, readByteNo-2);
```

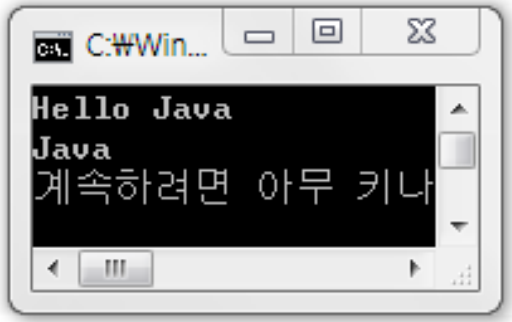
입력내용:

바이트 배열 내용 :



ByteToStringExample.java_바이트 배열을 문자열로 변환

```
1  public class ByteToStringExample {
2      public static void main(String[] args) {
3          byte[] bytes = { 72, 101, 108, 108, 111, 32, 74, 97, 118, 97  };
4
5          String str1 = new String(bytes);
6          System.out.println(str1);
7
8          String str2 = new String(bytes, 6, 4);
9          System.out.println(str2);
10     }
11 }
12
```



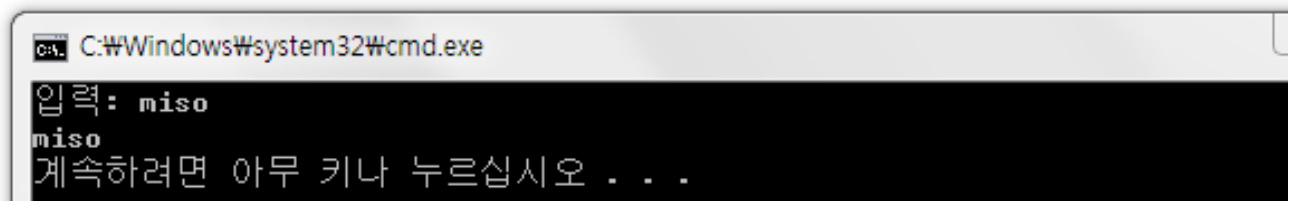
KeyboardToStringExample.java_바이트 배열을 문자열로 변환

```java
import java.io.IOException;

public class KeyboardToStringExample {
    public static void main(String[] args) throws IOException {
        byte[] bytes = new byte[100];

        System.out.print("입력: ");
        int readByteNo = System.in.read(bytes);

        String str = new String(bytes, 0, readByteNo-2);
        System.out.println(str);
    }
}
```



7절. String 클래스

❖ String 메소드

- 문자열의 추출, 비교, 찾기, 분리, 변환등과 같은 다양한 메소드 가짐
- 사용 빈도 높은 메소드

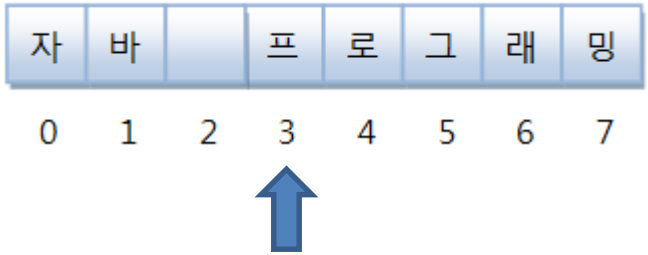
리턴타입	메소드명(매개변수)	설명
char	charAt(int index)	특정 위치의 문자 리턴
boolean	equals(Object anObject)	두 문자열을 비교
byte[]	getBytes()	byte[]로 리턴
byte[]	getBytes(Charset charset)	주어진 문자셋으로 인코딩한 byte[]로 리턴
int	indexOf(String str)	문자열내에서 주어진 문자열의 위치를 리턴
int	length()	총 문자의 수를 리턴
String	replace(CharSequence target, CharSequence replacement)	target 부분을 replacement 로 대치한 새로운 문자열을 리턴
String	substring(int beginIndex)	beginIndex 위치에서 끝까지 짤라낸 새로운 문자열을 리턴
String	substring(int beginIndex, int endIndex)	beginIndex 위치에서 endIndex 전까지 짤라낸 새로운 문자열을 리턴
String	toLowerCase()	알파벳 소문자로 변환한 새로운 문자열을 리턴
String	toUpperCase()	알파벳 대문자로 변환한 새로운 문자열을 리턴
String	trim()	앞뒤 공백을 제거한 새로운 문자열을 리턴
String	valueOf(int i) valueOf(double d)	기본 타입값을 문자열로 리턴

7절. String 클래스

문자 추출(charAt())

- 매개 값으로 주어진 인덱스의 문자 리턴

```
String subject = "자바 프로그래밍";
char charValue = subject.charAt(3);
```



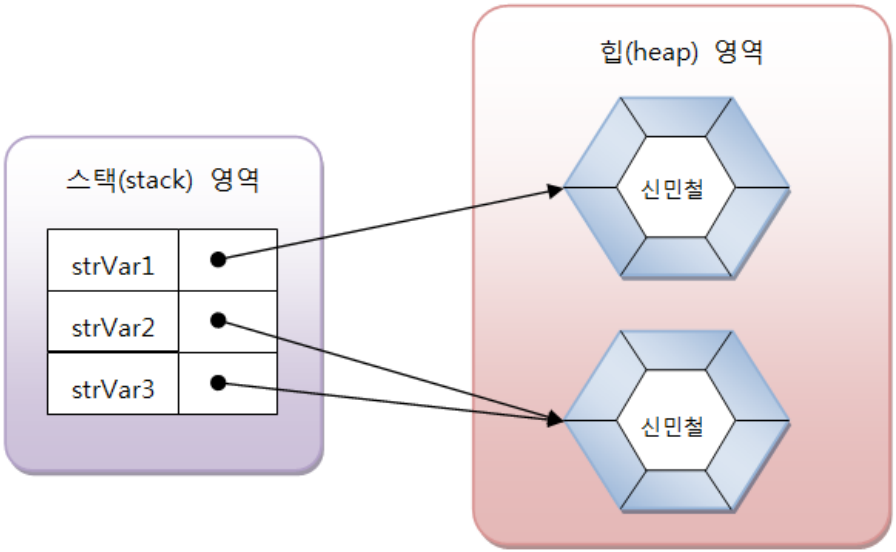
문자열 비교(equals())

- **문자열 비교할 때 == 연산자 사용하면 원하지 않는 결과 발생!**

```
String strVar1 = new String("신민철");
String strVar2 = "신민철";
String strVar3 = "신민철";
```

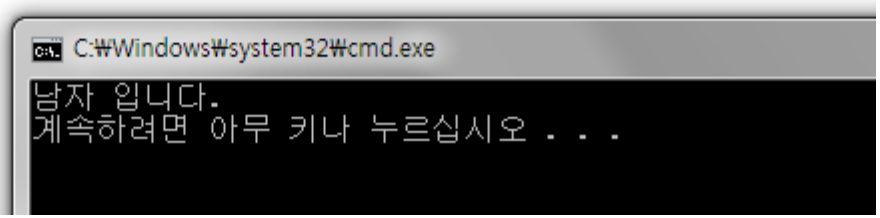
```
strVar1 == strVar2  → false
strVar2 == strVar3  → true
```

```
strVar1.equals(strVar2)  → true
strVar2.equals(strVar3)  → true
```



StringCharatExample.java_주민번호에서 남자와 여자를 구분하는 방법

```java
public class StringCharAtExample {
    public static void main(String[] args) {
        String ssn = "010624-1230123";
        char sex = ssn.charAt(7);
        switch (sex) {
            case '1':
            case '3':
                System.out.println("남자 입니다.");
                break;
            case '2':
            case '4':
                System.out.println("여자 입니다.");
                break;
        }
    }
}
```

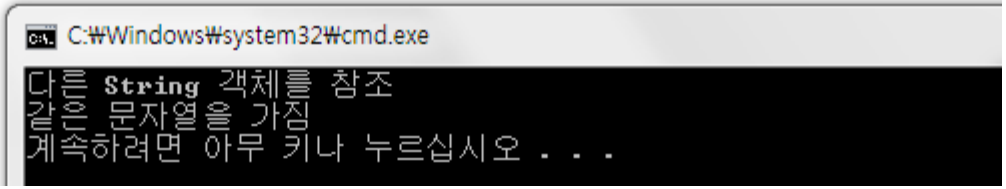


StringEqualsExample.java_문자열 비교

```java
public class StringEqualsExample {
    public static void main(String[] args) {
        String strVar1 = new String("신민철");
        String strVar2 = "신민철";

        if(strVar1 == strVar2) {
            System.out.println("같은 String 객체를 참조");
        } else {
            System.out.println("다른 String 객체를 참조");
        }

        if(strVar1.equals(strVar2)) {
            System.out.println("같은 문자열을 가짐");
        } else {
            System.out.println("다른 문자열을 가짐");
        }
    }
}
```



7절. String 클래스

- **바이트 배열로 변환(getBytes())**
  - **시스템의 기본 문자셋으로 인코딩된 바이트 배열 얻기**

```
byte[] bytes = "문자열".getBytes();
```

  - **특정 문자셋으로 인코딩 된 바이트 배열 얻기**

```
try {
   byte[] bytes = "문자열".getBytes("EUC-KR");
   byte[] bytes = "문자열".getBytes("UTF-8");
} catch (UnsupportedEncodingException e) {
}
```

[참고] 디코딩

```
String str = new String(byte[] bytes, String charsetName);
```



StringGetBytesExample.java_바이트 배열로 변환

```java
import java.io.UnsupportedEncodingException;

public class StringGetBytesExample {
    public static void main(String[] args) {
        String str = "안녕하세요";

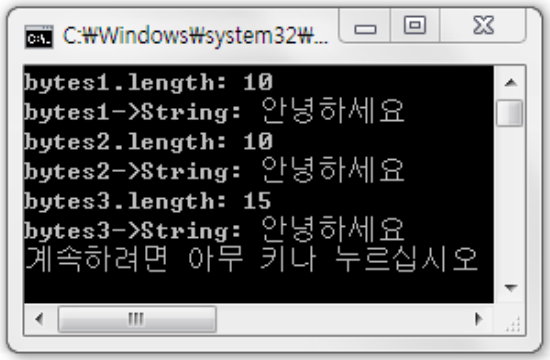
        byte[] bytes1 = str.getBytes();
        System.out.println("bytes1.length: " + bytes1.length);
        String str1 = new String(bytes1);
        System.out.println("bytes1->String: " + str1);

        try {

            byte[] bytes2 = str.getBytes("EUC-KR");
            System.out.println("bytes2.length: " + bytes2.length);
            String str2 = new String(bytes2, "EUC-KR");
            System.out.println("bytes2->String: " + str2);


            byte[] bytes3 = str.getBytes("UTF-8");
            System.out.println("bytes3.length: " + bytes3.length);
            String str3 = new String(bytes3, "UTF-8");
            System.out.println("bytes3->String: " + str3);

        } catch (UnsupportedEncodingException e) {
            e.printStackTrace();
        }
    }
}
```

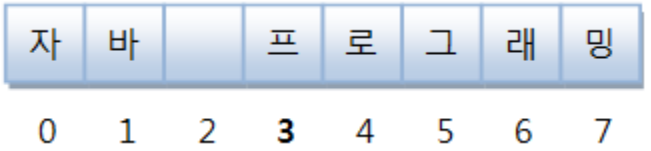




7절. String 클래스

- 문자열 찾기(indexOf())
  - 매개값으로 주어진 문자열이 시작되는 인덱스 리턴
  - 주어진 문자열이 포함되어 있지 않으면 -1 리턴

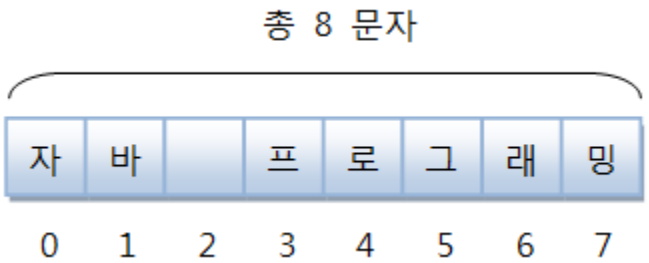
```
String subject = "자바 프로그래밍";
int index = subject.indexOf("프로그래밍");
```



  - 특정 문자열이 포함되어 있는지 여부 따라 실행 코드 달리할 때 사용

- 문자열 길이(length()) – 공백도 문자에 포함

```
String subject = "자바 프로그래밍";
int length = subject.length();
```

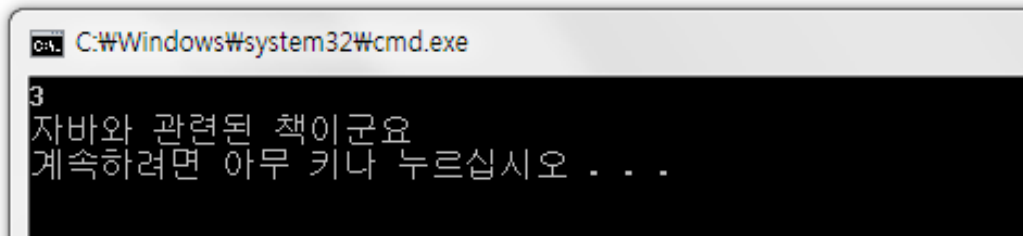


StringIndexOfExample.java_문자열 포함 여부 조사

```java
public class StringIndexOfExample {
    public static void main(String[] args) {
        String subject = "자바 프로그래밍";

        int location = subject.indexOf("프로그래밍");
        System.out.println(location);

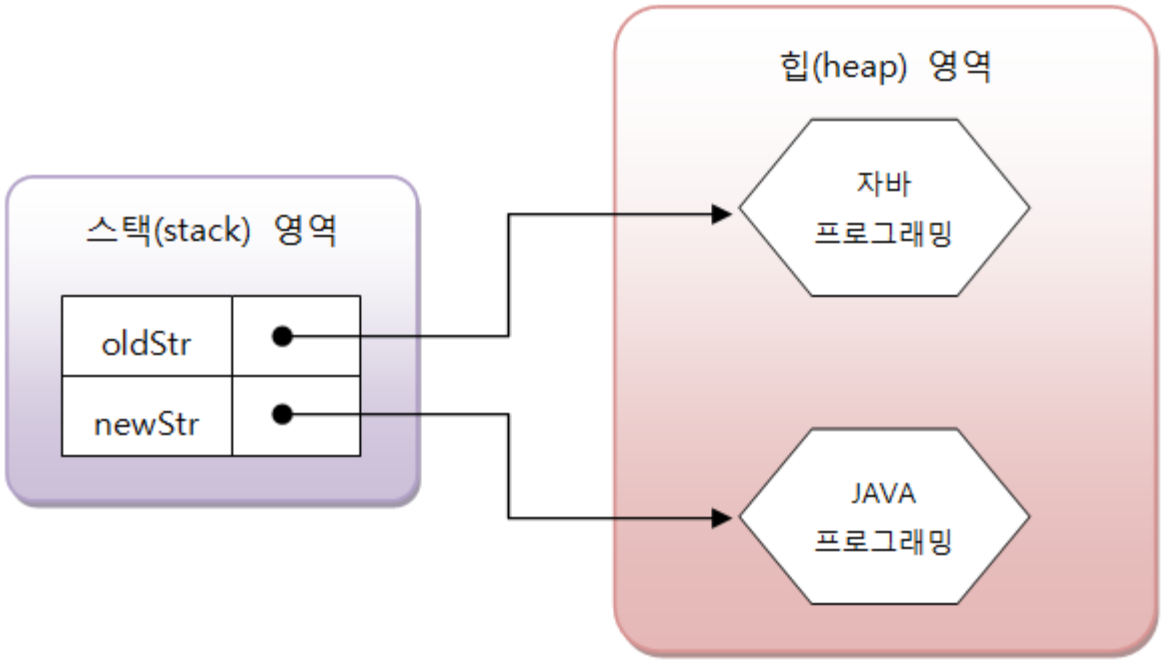
        if(subject.indexOf("자바") != -1) {
            System.out.println("자바와 관련된 책이군요");
        } else {
            System.out.println("자바와 관련없는 책이군요");
        }
    }
}
```



7절. String 클래스

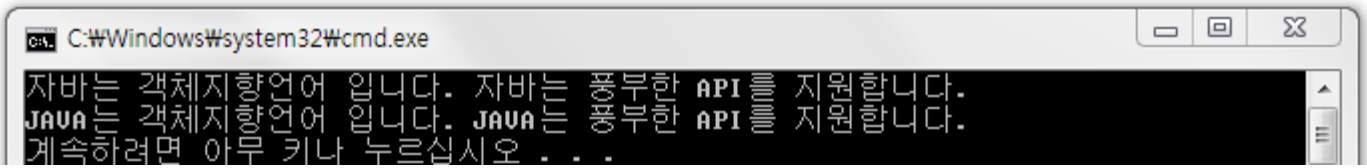
- 문자열 대치(replace())
  - 첫 번째 매개값인 문자열 찾음
  - 두 번째 매개값인 문자열로 대치
  - 새로운 문자열 리턴

```
String oldStr = "자바 프로그래밍";
String newStr = oldStr.replace("자바", "JAVA");
```



StringReplaceExample.java_문자열 대치하기

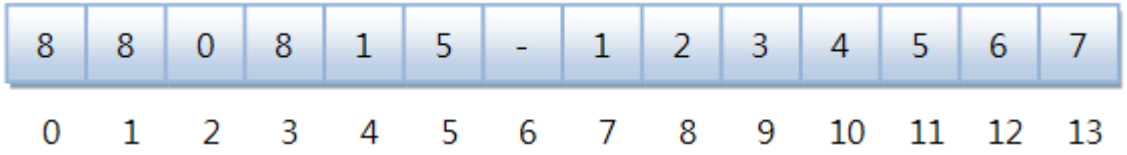
```
1 public class StringReplaceExample {
2     public static void main(String[] args) {
3         String oldStr = "자바는 객체지향언어 입니다. 자바는 풍부한 API를 지원합니다.";
4         String newStr = oldStr.replace("자바", "JAVA");
5
6         System.out.println(oldStr);
7         System.out.println(newStr);
8     }
9 }
10
11
```





7절. String 클래스

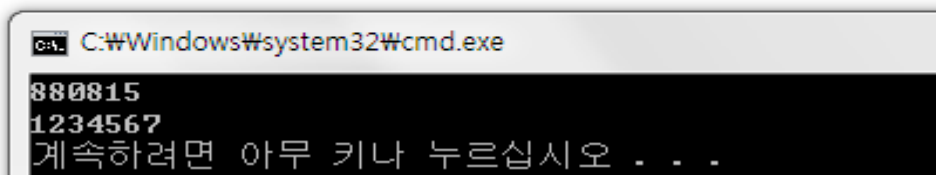
- 문자열 잘라내기(substring())
  - substring(int beginIndex, int endIndex)
    - 주어진 시작과 끝 인덱스 사이의 문자열 추출
  - substring(int beginIndex)
    - 주어진 인덱스 이후부터 끝까지 문자열 추출




```
String ssn  = "880815-1234567";
String firstNum  = ssn.substring(0,  6);
String secondNum  = ssn.substring(7);
```

StringSubstringExample.java_문자열 추출하기

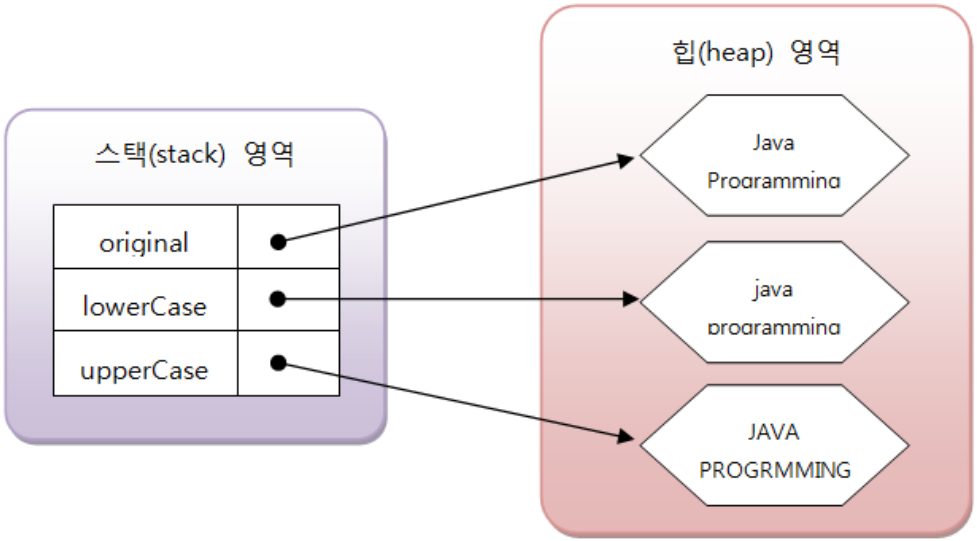
```
1  public class StringSubstringExample {
2      public static void main(String[] args) {
3          String ssn = "880815-1234567 ";
4
5          String firstNum = ssn.substring(0, 6);
6          System.out.println(firstNum);
7
8          String secondNum = ssn.substring(7);
9          System.out.println(secondNum);
10     }
11 }
12
13
```



7절. String 클래스

- **알파벳 소·대문자 변경 (toLowerCase(), toUpperCase())**

```
String original = "Java Programming";
String lowerCase = original.toLowerCase();
String upperCase = original.toUpperCase();
```



StringToLowerUpperExample.java_전부 소문자 또는 대문자로 변경

```java
public class StringToLowerUpperCaseExample {
    public static void main(String[] args) {
        String str1 = "Java Programming";
        String str2 = "JAVA Programming";

        System.out.println(str1.equals(str2));

        String lowerStr1 = str1.toLowerCase();
        String lowerStr2 = str2.toLowerCase();
        System.out.println(lowerStr1.equals(lowerStr2));

        System.out.println(str1.equalsIgnoreCase(str2));
    }
}
```

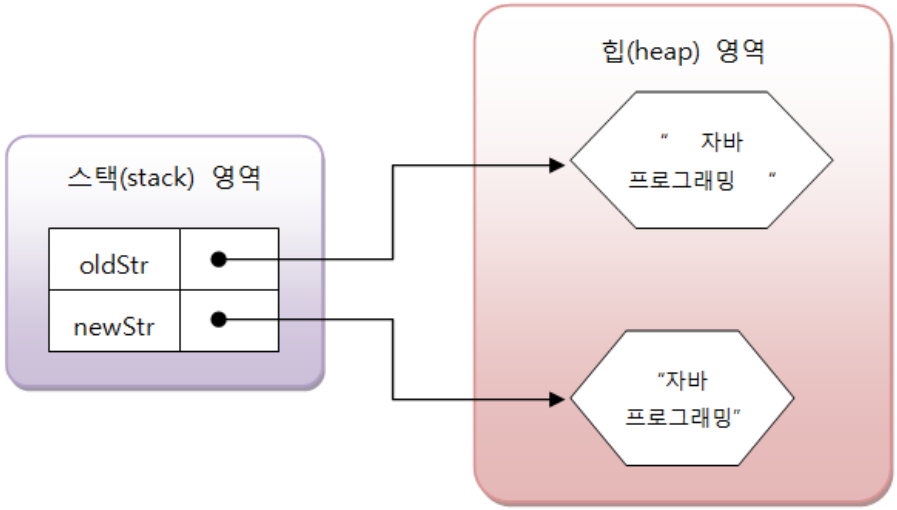
```
C:₩Windows₩system32₩cmd.exe
false
true
true
계속하려면 아무 키나 누르십시오 . . .
```



7절. String 클래스

❖ 문자열 앞뒤 공백 잘라내기(trim())

```
String oldStr = "    자바 프로그래밍    ";
String newStr = oldStr.trim();
```



StringTrimExample.java_앞뒤 공백 제거

```java
public class StringTrimExample {
    public static void main(String[] args) {
        String tel1 = "  02";
        String tel2 = "123   ";
        String tel3 = "   1234   ";

        String tel = tel1.trim() + tel2.trim() + tel3.trim();
        System.out.println(tel);
    }
}
```



7절. String 클래스

- 문자열 변환(valueOf())
  - 기본 타입의 값을 문자열로 변환

```
static String valueOf(boolean b)
static String valueOf(char c)
static String valueOf(int i)
static String valueOf(long l)
static String valueOf(double d)
static String valueOf(float f)
```



StringValueOfExample.java_기본 타입 값을 문자열로 변환

```java
public class StringValueOfExample {
    public static void main(String[] args) {
        String str1 = String.valueOf(10);
        String str2 = String.valueOf(10.5);
        String str3 = String.valueOf(true);

        System.out.println(str1);
        System.out.println(str2);
        System.out.println(str3);
    }
}
```



8절. StringTokenizer 클래스

❖ 문자열 분리 방법

- String의 split() 메소드 이용
- java.util.StringTokenizer 클래스 이용

❖ String의 split()

- 정규표현식을 구분자로 해서 부분 문자열 분리
- 배열에 저장하고 리턴

```
홍길동&이수홍,박연수,김자바-최명호
```

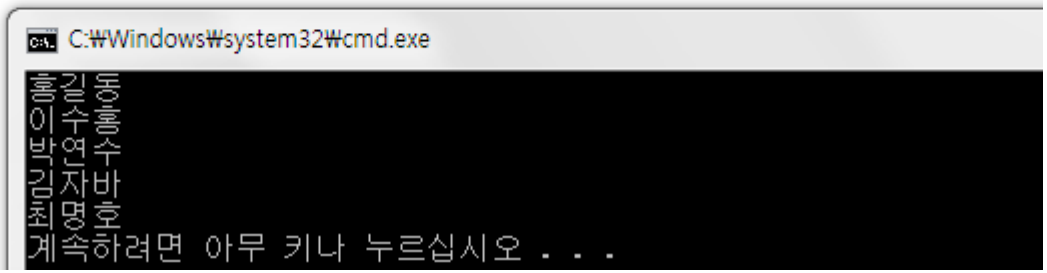
```
String[] names = text.split("&|,|-");
```



StringSplitExample.java_문자열 분리

```java
public class StringSplitExample {
    public static void main(String[] args) {
        String text = "홍길동&이수홍,박연수,김자바-최명호";
        String[] names = text.split("&|,|-");

        for(String name : names) {
            System.out.println(name);
        }
    }
}
```



8절. StringTokenizer 클래스

❖ StringTokenizer 클래스

```
String text = "홍길동/이수홍/박연수";
StringTokenizer st = new StringTokenizer(text, "/");
```

```
while( st.hasMoreTokens() ) {
   String token = st.nextToken();
   System.out.println(token);
}
```

메소드		설명
int	countTokens()	꺼내지 않고 남아있는 토큰의 수
boolean	hasMoreTokens()	남아 있는 토큰이 있는지 여부
String	nextToken()	토큰을 하나씩 꺼내옴



StringTokenizerExample.java_문자열 분리

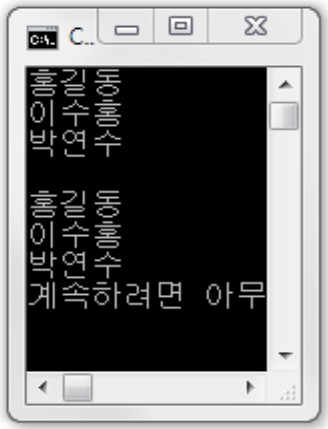
```java
import java.util.StringTokenizer;

public class StringTokenizerExample {
    public static void main(String[] args) {
        String text = "홍길동/이수홍/박연수";

        //how1: 전체 토큰 수를 얻어 for문으로 루핑
        StringTokenizer st = new StringTokenizer(text, "/");
        int countTokens = st.countTokens();
        for(int i=0; i<countTokens; i++) {
            String token = st.nextToken();
            System.out.println(token);
        }

        System.out.println();

        //how2: 남아 있는 토큰를 확인하고 while문으로 루핑
        st = new StringTokenizer(text, "/");
        while( st.hasMoreTokens() ) {
            String token = st.nextToken();
            System.out.println(token);
        }
    }
}
```

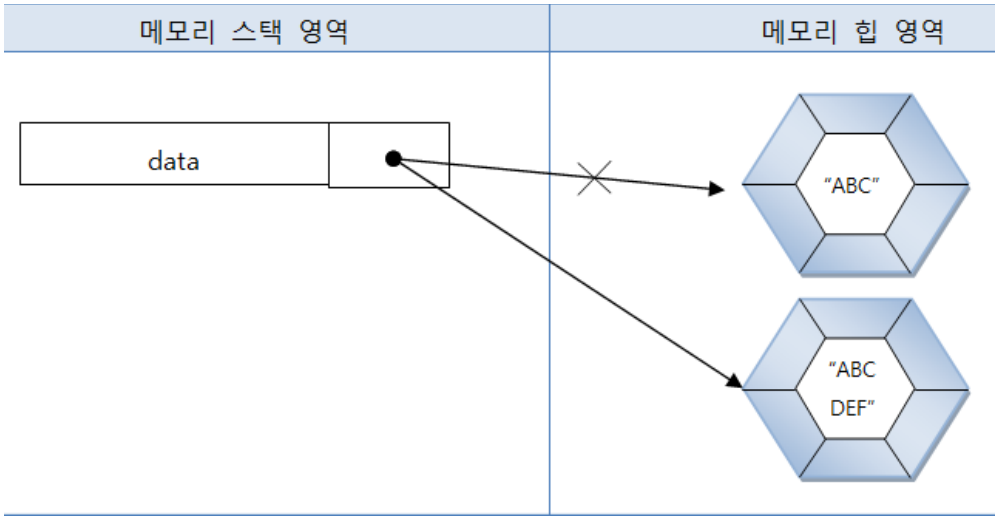


9절. StringBuffer, StringBuilder 클래스

❖ 문자열 결합 연산자 +

- String 은 내부의 문자열 수정 불가 -> 대치된 새로운 문자열 리턴

```
String data = "ABC";
data += "DEF";
```



9절. StringBuffer, StringBuilder 클래스

❖ StringBuffer, StringBuilder (p.514~516)

- 버퍼(buffer:데이터를 임시로 저장하는 메모리)에 문자열 저장
- 버퍼 내부에서 추가, 수정, 삭제 작업 가능
- 멀티 스레드환경: StringBuffer 사용
- 단일 스레드환경: StringBuilder 사용

```
StringBuilder sb = new StringBuilder();
StringBuilder sb = new StringBuilder(16);
StringBuilder sb = new StringBuilder("Java");
```

메소드
append(...)
insert(int offset, ...)
delete(int start, int end)
deleteCharAt(int index)
replace(int start, int end, String str)
StringBuilder reverse()
setCharAt(int index, char ch)



StringBuilderExample.java_StringBuilder에서 문자열 조작

```java
public class StringBuilderExample {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder();

        sb.append("Java ");
        sb.append("Program Study");
        System.out.println(sb.toString());

        sb.insert(4, "2");
        System.out.println(sb.toString());

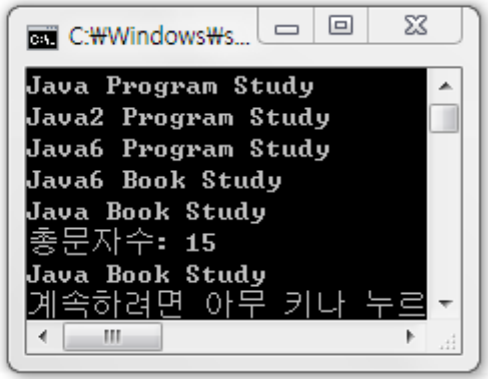
        sb.setCharAt(4, '6');
        System.out.println(sb.toString());

        sb.replace(6, 13, "Book");
        System.out.println(sb.toString());

        sb.delete(4, 5);
        System.out.println(sb.toString());

        int length = sb.length();
        System.out.println("총문자수: " + length);

        String result = sb.toString();
        System.out.println(result);
    }
}
```



10절. 정규 표현식과 Pattern 클래스

❖ 정규 표현식(Regular Expression) 작성 방법

- 문자열이 정해져 있는 형식으로 구성되어 있는지 검증할 때 사용
  - Ex) 이메일, 전화번호, 비밀번호 등

- 문자 또는 숫자 기호와 반복 기호가 결합된 문자열

기호	설명		
[ ]	한 개의 문자	[abc]	a, b, c 중 하나의 문자
		[^abc]	a, b, c 이외의 하나의 문자
		[a-zA-Z]	a~z, A~Z 중 하나의 문자
₩d	한 개의 숫자, [0-9]와 동일		
₩s	공백		
₩w	한 개의 알파벳 또는 한 개의 숫자, [a-zA-Z_0-9]와 동일		
?	없음 또는 한 개		
*	없음 또는 한 개 이상		
+	한 개 이상		
{n}	정확히 n 개		
{n,}	최소한 n 개		
{n, m}	n 개에서부터 m 개까지		
( )	그룹핑		

10절. 정규 표현식과 Pattern 클래스

❖ Pattern 클래스

- 정규 표현식으로 문자열을 검증하는 역할
  - **결과는 boolean 타입 !!!**

```
boolean result = Pattern.matches("정규식", "입력된 문자열");
```

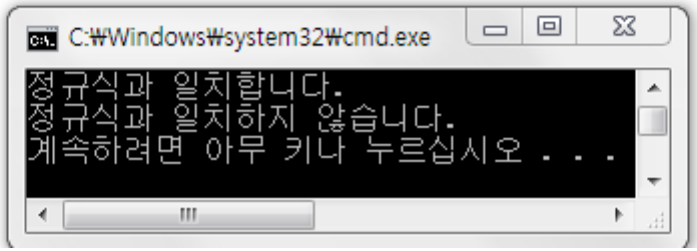


PatternExample.java_문자열 검증하기

```java
import java.util.regex.Pattern;

public class PatternExample {
    public static void main(String[] args) {
        String regExp = "(02|010)-\\d{3,4}-\\d{4}";
        String data = "010-123-4567";
        boolean result = Pattern.matches(regExp, data);
        if(result) {
            System.out.println("정규식과 일치합니다.");
        } else {
            System.out.println("정규식과 일치하지 않습니다.");
        }

        regExp = "\\w+@\\w+\\.\\w+(\\.\\w+)?";
        data = "angel@navercom";
        result = Pattern.matches(regExp, data);
        if(result) {
            System.out.println("정규식과 일치합니다.");
        } else {
            System.out.println("정규식과 일치하지 않습니다.");
        }
    }
}
```



11절. Arrays 클래스

❖ Arrays

- 배열 조작 기능을 가지고 있는 클래스 – 배열 복사, 항목 정렬, 항목 검색
- 제공하는 정적 메소드

리턴타입	메소드 이름	설명
int	binarySearch(배열, 찾는값)	전체 배열 항목에서 찾는값이 있는 인덱스 리턴
타겟배열	copyOf(원본배열, 복사할길이)	원본배열의 0 번 인덱스에서 복사할 길이만큼 복사한 배열 리턴, 복사할 길이는 원본배열의 길이보다 크도 되며, 타겟배열의 길이가 된다.
타겟배열	copyOfRange(원본배열, 시작인덱스, 끝인덱스)	원본배열의 시작인덱스에서 끝인덱스까지 복사한 배열 리턴
boolean	deepEquals(배열, 배열)	두 배열의 깊은 비교(중첩 배열의 항목까지 비교)
boolean	equals(배열, 배열)	얕은 비교(중첩 배열의 항목은 비교하지 않음)
void	fill(배열, 값)	전체 배열 항목에 동일한 값을 저장
void	fill(배열, 시작인덱스, 끝인덱스, 값)	시작인텍스부터 끝인덱스까지의 항목에만 동일한 값을 저장
void	sort(배열)	배열의 전체 항목을 올림차순으로 정렬

11절. Arrays 클래스

- 배열 복사
  - Arrays.copyOf(원본배열, 복사할 길이)
    - 0 ~ (복사할 길이-1)까지 항목 복사
    - 복사할 길이는 원본 배열의 길이보다 커도 되며 타겟 배열의 길이

```
char[] arr1 = {'J', 'A', 'V', 'A'};
char[] arr2 = Arrays.copyOf(arr1,  arr1.length);
```

  - copyOfRange(원본 배열, 시작 인덱스, 끝 인덱스)
    - 시작인덱스 ~ (끝 인덱스-1)까지 항목 복사

```
char[] arr1 = {'J', 'A', 'V', 'A'};
char[] arr2 = Arrays.copyOfRange(arr1,  1, 3);
```

  - System.arraycopy()

```
System.arraycopy(Object src,  int srcPos,  Object dest,  int destPos,  int length)
                 원본배열     원본시작인덱스   타겟배열     타겟시작인덱스    복사개수
```

ArrayCopyExample.java_배열 복사

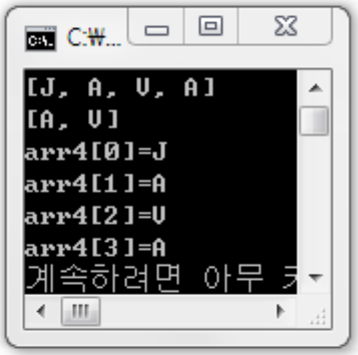
```java
import java.util.Arrays;

public class ArrayCopyExample {
    public static void main(String[] args) {
        char[] arr1 = {'J', 'A', 'V', 'A'};

        //방법1
        char[] arr2 = Arrays.copyOf(arr1, arr1.length);
        System.out.println(Arrays.toString(arr2));

        //방법2
        char[] arr3 = Arrays.copyOfRange(arr1, 1, 3);
        System.out.println(Arrays.toString(arr3));

        //방법3
        char[] arr4 = new char[arr1.length];
        System.arraycopy(arr1, 0, arr4, 0, arr1.length);
        for(int i=0; i<arr4.length; i++) {
            System.out.println("arr4[" + i + "]=" + arr4[i]);
        }
    }
}
```



11절. Arrays 클래스

- 배열 항목 비교
  - Arrays.equals(배열, 배열) – 1차 항목의 값만 비교
  - Arrays.deepEquals(배열, 배열) – 중첩된 배열의 항목까지 비교
- 배열 항목 정렬
  - Arrays.sort(배열)– 항목 오름차 순으로 정렬
    - 기본 타입이거나 String 배열 자동 정렬
  - 사용자 정의 클래스 배열은 Comparable 인터페이스를 구현해야만 정렬
- 배열 항목 검색
  - 특정 값 위치한 인덱스 얻는 것
  - Arrays.sort(배열)로 먼저 정렬
  - Arrays.binarySearch(배열, 찾는 값) 메소드로 항목을 찾아야



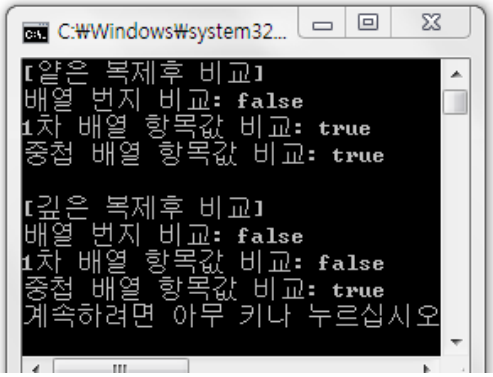
EqualsExample.java_배열 비교

```java
import java.util.Arrays;

public class EqualsExample {
    public static void main(String[] args) {
        int[][] original = { {1,2}, {3,4} };

        //얕은 복사후 비교
        System.out.println("[얕은 복제후 비교]");
        int[][] cloned1 = Arrays.copyOf(original, original.length);
        System.out.println("배열 번지 비교: " + original.equals(cloned1));
        System.out.println("1차 배열 항목값 비교: " + Arrays.equals(original, cloned1));
        System.out.println("중첩 배열 항목값 비교: " + Arrays.deepEquals(original, cloned1));

        //깊은 복사후 비교
        System.out.println("\n[깊은 복제후 비교]");
        int[][] cloned2 = Arrays.copyOf(original, original.length);
        cloned2[0] = Arrays.copyOf(original[0], original[0].length);
        cloned2[1] = Arrays.copyOf(original[1], original[1].length);
        System.out.println("배열 번지 비교: " + original.equals(cloned2));
        System.out.println("1차 배열 항목값 비교: " + Arrays.equals(original, cloned2));
        System.out.println("중첩 배열 항목값 비교: " + Arrays.deepEquals(original, cloned2));
    }
}
```



12절. 포장(Wrapper) 클래스

❖ 포장(Wrapper) 객체란?

- 기본 타입(byte, char, short, int, long, float, double, boolean) 값을 내부에 두고 포장하는 객체
- 기본 타입의 값은 외부에서 변경 불가

기본 타입	포장 클래스
byte	Byte
char	Character
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean



12절. 포장(Wrapper) 클래스

❖ 박싱(Boxing)과 언박싱(Unboxing)

- 박싱(Boxing): 기본 타입의 값을 포장 객체로 만드는 과정
- 언박싱(Unboxing): 포장 객체에서 기본 타입의 값을 얻어내는 과정
- 박싱 하는 방법
  - 생성자 이용

기본 타입의 값을 줄 경우	문자열을 줄 경우
Byte obj = new Byte(10);	Byte obj = new Byte("10");
Character obj = new Character('가');	
Short obj = new Short(100);	Short obj = new Short("100");
Integer obj = new Integer(1000);	Integer obj = new Integer("1000");
Long obj = new Long(10000);	Long obj = new Long("10000");
Float obj = new Float(2.5F);	Float obj = new Float("2.5F");
Double obj = new Double(3.5);	Double obj = new Double("3.5");
Boolean obj = new Boolean(true);	Boolean obj = new Boolean("true");

  - valueOf() 메소드 이용

```
Integer obj = Integer.valueOf(1000);
Integer obj = Integer.valueOf("1000");
```

12절. 포장(Wrapper) 클래스

- 언박싱 코드
  - 각 포장 클래스마다 가지고 있는 클래스 호출
  - 기본 타입명 + Value()

기본 타입의 값을 이용		
byte	num	= obj.byteValue();
char	ch	= obj.charValue();
short	num	= obj.shortValue();
int	num	= obj.intValue();
long	num	= obj.longValue();
float	num	= obj.floatValue();
double	num	= obj.doubleValue();
boolean	bool	= obj.booleanValue();



BoxingUnBoxingExample.java_기본 타입의 값을 박싱하고 언박싱하기

```java
public class BoxingUnBoxingExample {
    public static void main(String[] args) {
        //Boxing
        Integer obj1 = new Integer(100);
        Integer obj2 = new Integer("200");
        Integer obj3 = Integer.valueOf("300");

        //Unboxing
        int value1 = obj1.intValue();
        int value2 = obj2.intValue();
        int value3 = obj3.intValue();

        System.out.println(value1);
        System.out.println(value2);
        System.out.println(value3);
    }
}
```



12절. 포장(Wrapper) 클래스

❖ 자동 박싱과 언박싱

- **자동 박싱 - 포장 클래스 타입에 기본값이 대입될 경우 발생**

```
Integer obj = 100;     //자동 박싱
```

```
List<Integer> list = new ArrayList<Integer>();
list.add(200);    //자동 박싱
```

- **자동 언박싱 - 기본 타입에 포장 객체가 대입될 경우 발생**

```
Integer obj = new Integer(200);
int value1 = obj;              //자동 언박싱
int value2 = obj + 100;     //자동 언박싱
```

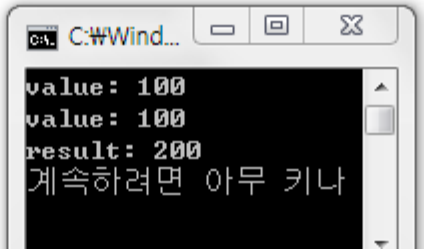


AutoBoxingUnBoxingExample.java_자동 박싱하고 언박싱하기

```java
public class AutoBoxingUnBoxingExample {
    public static void main(String[] args) {
        //자동 Boxing
        Integer obj = 100;
        System.out.println("value: " + obj.intValue());

        //대입시 자동 Unboxing
        int value = obj;
        System.out.println("value: " + value);

        //연산시 자동 Unboxing
        int result = obj + 100;
        System.out.println("result: " + result);
    }
}
```



12절. 포장(Wrapper) 클래스

❖ 문자열을 기본 타입 값으로 변환

- parse + 기본타입 명 → 정적 메소드

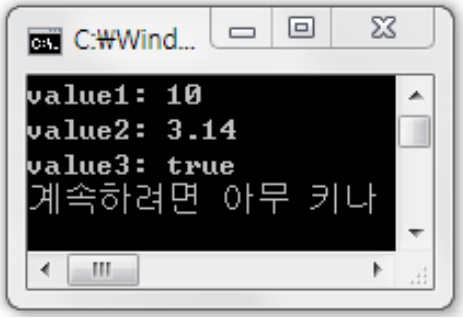
기본 타입의 값을 이용		
byte	num	= Byte.parseByte("10");
short	num	= Short.parseShort("100");
int	num	= Integer.parseInt("1000");
long	num	= Long.parseLong("10000");
float	num	= Float.parseFloat("2.5F");
double	num	= Double.parseDouble("3.5");
boolean	bool	= Boolean.parseBoolean("true");

❖ 포장값 비교

- 포장 객체는 내부 값을 비교하기 위해 ==와 != 연산자 사용 불가
- 값을 언박싱해 비교하거나, equals() 메소드로 내부 값 비교할 것

StringToPrimitiveValueExample.java_문자열을 기본 타입 값으로 변환

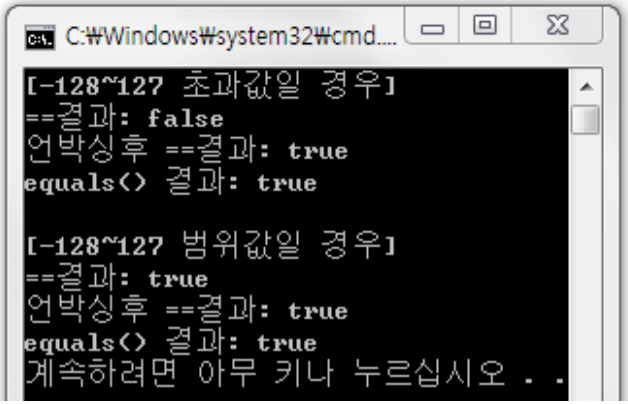
```
1 public class StringToPrimitiveValueExample {
2     public static void main(String[] args) {
3         int value1 = Integer.parseInt("10");
4         double value2 = Double.parseDouble("3.14");
5         boolean value3 = Boolean.parseBoolean("true");
6 
7         System.out.println("value1: " + value1);
8         System.out.println("value2: " + value2);
9         System.out.println("value3: " + value3);
10    }
11 }
12
```



ValueCompareExample.java_포장 객체 비교

```java
public class ValueCompareExample {
    public static void main(String[] args) {
        System.out.println("[-128~127 초과값일 경우]");
        Integer obj1 = 300;
        Integer obj2 = 300;
        System.out.println("==결과: " + (obj1 == obj2));
        System.out.println("언박싱후 ==결과: " + (obj1.intValue() == obj2.intValue()));
        System.out.println("equals() 결과: " + obj1.equals(obj2));
        System.out.println();

        System.out.println("[-128~127 범위값일 경우]");
        Integer obj3 = 10;
        Integer obj4 = 10;
        System.out.println("==결과: " + (obj3 == obj4));
        System.out.println("언박싱후 ==결과: " + (obj3.intValue() == obj4.intValue()));
        System.out.println("equals() 결과: " + obj3.equals(obj4));
    }
}
```



13절. Math, Random 클래스

❖ Math 클래스 (예제는 p.533~536 참고)

- 수학 계산에 사용할 수 있는 정적 메소드 제공

메소드	설명	예제 코드	리턴값
int abs(int a) double abs(double a)	절대값	int v1 = Math.abs(-5); double v2 = Math.abs(-3.14);	v1 = 5 v2 = 3.14
double ceil(double a)	올림값	double v3 = Math.ceil(5.3); double v4 = Math.ceil(-5.3);	v3 = 6.0 v4 = -5.0
double floor(double a)	버림값	double v5 = Math.floor(5.3); double v6 = Math.floor(-5.3);	v5 = 5.0 v6 = -6.0
int max(int a, int b) double max(double a, double b)	최대값	int v7 = Math.max(5, 9); double v8 = Math.max(5.3, 2.5);	v7 = 9 v8 = 5.3
int min(int a, int b) double min(double a, double b)	최소값	int v9 = Math.min(5, 9); double v10 = Math.min(5.3, 2.5);	v9 = 5 v10 = 2.5
double random()	랜덤값	double v11 = Math.random();	0.0<= v11<1.0
double rint(double a)	가까운 정수의 실수값	double v12 = Math.rint(5.3); double v13 = Math.rint(5.7);	v12 = 5.0 v13 = 6.0
long round(double a)	반올림값	long v14 = Math.round(5.3); long v15 = Math.round(5.7);	v14 = 5 v15 = 6

MathExample.java_Math의 수학 메소드

```java
public class MathExample {
    public static void main(String[] args)
        int v1 = Math.abs(-5);
        double v2 = Math.abs(-3.14);
        System.out.println("v1=" + v1);
        System.out.println("v2=" + v2);

        double v3 = Math.ceil(5.3);
        double v4 = Math.ceil(-5.3);
        System.out.println("v3=" + v3);
        System.out.println("v4=" + v4);

        double v5 = Math.floor(5.3);
        double v6 = Math.floor(-5.3);
        System.out.println("v5=" + v5);
        System.out.println("v6=" + v6);

        int v7 = Math.max(5, 9);
        double v8 = Math.max(5.3, 2.5);
        System.out.println("v7=" + v7);
        System.out.println("v8=" + v8);

        int v9 = Math.min(5, 9);
        double v10 = Math.min(5.3, 2.5);
        System.out.println("v9=" + v9);
        System.out.println("v10=" + v10);

        double v11 = Math.random();
        System.out.println("v11=" + v11);

        double v12 = Math.rint(5.3);
        double v13 = Math.rint(5.7);
        System.out.println("v12=" + v12);
        System.out.println("v13=" + v13);

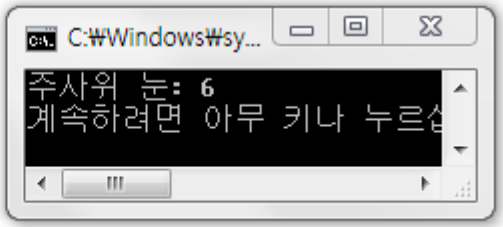
        long v14 = Math.round(5.3);
        long v15 = Math.round(5.7);
        System.out.println("v14=" + v14);
        System.out.println("v15=" + v15);

        double value = 12.3456;
        double temp1 = value * 100;
        long temp2 = Math.round(temp1);
        double v16 = temp2 / 100.0;
        System.out.println("v16=" + v16);
    }
}
```

```
v1=5
v2=3.14
v3=6.0
v4=-5.0
v5=5.0
v6=-6.0
v7=9
v8=5.3
v9=5
v10=2.5
v11=0.16146610660316774
v12=5.0
v13=6.0
v14=5
v15=6
v16=12.35
```

MathRandomExample.java_임의의 주사위의 눈 얻기

```
1 public class MathRandomExample {
2     public static void main(String[] args) {
3         int num = (int) (Math.random()*6) + 1;
4         System.out.println("주사위 눈: " + num);
5     }
6 }
7
```



13절. Math, Random 클래스

❖ Random 클래스

- boolean, int, long, float, double 난수 입수 가능
- 난수를 만드는 알고리즘에 사용되는 종자값(seed) 설정 가능
  - 종자값이 같으면 같은 난수

- Random 클래스로 부터 Random객체 생성하는 방법

생성자	설명
Random()	호출시 마다 다른 종자값(현재시간 이용)이 자동 설정된다.
Random(long seed)	매개값으로 주어진 종자값이 설정된다.

- Random 클래스가 제공하는 메소드

리턴값	메소드(매개변수)	설명
boolean	nextBoolean()	boolean 타입의 난수를 리턴
double	nextDouble()	double 타입의 난수를 리턴(0.0<= ~ < 1.0)
int	nextInt()	int 타입의 난수를 리턴(-2^{32}<= ~ <=2^{32}-1);
int	nextInt(int n)	int 타입의 난수를 리턴(0<= ~ <n)

RandomExample.java_로또 번호 얻기

```java
import java.util.Arrays;
import java.util.Random;

public class RandomExample {
    public static void main(String[] args) {
        //선택번호
        int[] selectNumber = new int[6];
        Random random = new Random(3);
        System.out.print("선택 번호: ");
        for(int i=0; i<6; i++) {
            selectNumber[i] = random.nextInt(45) + 1;
            System.out.print(selectNumber[i] + " ");
        }
        System.out.println();

        //당첨번호
        int[] winningNumber = new int[6];
        random = new Random(5);
        System.out.print("당첨 번호: ");
        for(int i=0; i<6; i++) {
            winningNumber[i] = random.nextInt(45) + 1;
            System.out.print(winningNumber[i] + " ");
        }
        System.out.println();

        //당첨여부
        Arrays.sort(selectNumber);
        Arrays.sort(winningNumber);
        boolean result = Arrays.equals(selectNumber, winningNumber);
        System.out.print("당첨 여부: ");
        if(result) {
            System.out.println("1등에 당첨되셨습니다.");
        } else {
            System.out.println("당첨되지 않았습니다.");
        }
    }
}
```

```
선택 번호: 15 21 16 17 34 28
당첨 번호: 18 38 45 15 22 36
당첨 여부: 당첨되지 않았습니다.
계속하려면 아무 키나 누르십시오 .
```